3

Exploring LLMs as a Powerful AI Engine

In the previous chapter, we saw the structure of a transformer, how it is trained, and what makes it so powerful. The transformer is the seed of this revolution in **natural language processing** (**NLP**), and today's **large language models** (**LLMs**) are all based on transformers trained at scale. In this chapter, we will see what happens when we train huge transformers (more than 100 billion parameters) with giant datasets. We will focus on how to enable this training at scale, how to fine-tune similar modern ones, how to get more manageable models, and how to extend them to multimodal data. At the same time, we will also see what the limitations of these models are and what techniques are used to try to overcome these limitations.

In this chapter, we'll be covering the following topics:

- Discovering the evolution of LLMs
- Instruction tuning, fine-tuning, and alignment
- Exploring smaller and more efficient LLMs
- Exploring multimodal models
- Understanding hallucinations and ethical and legal issues
- Prompt engineering

Technical requirements

Most of this code can be run on a CPU, but it is preferable to run it on a GPU. The code is written in PyTorch and uses standard libraries for the

most part (PyTorch, Hugging Face Transformers, and so on). The code can be found on GitHub: **https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr3**.

Discovering the evolution of LLMs

An LLM is a transformer (although different architectures are beginning to emerge today). In general, an LLM is defined as a model that has more than 10 billion parameters. Although this number may seem arbitrary, some properties emerge with scale. These models are designed to understand and generate human language, and over time, they have acquired the ability to generate code and more. To achieve this beyond parameter size, they are trained with a huge amount of data. Today's LLMs are almost all trained on **next-word prediction** (**autoregressive language modeling**).

Parameter growth has been motivated in the transformer field by different aspects:

- **Learnability**: According to the scaling law, more parameters should lead to greater capabilities and a greater understanding of nuances and complexities in the data
- **Expressiveness**: The model can express more complex functions, thus increasing the ability to generalize and reducing the risk of overfitting
- **Memory**: A larger number of parameters allows for internalizing more knowledge (information, entities, differences in topics)

In the next subsections, we will discuss in detail all these elements to explain what is happening in the transition from the transformer to the LLM.

The scaling law

It may seem surprising that such large models are trained with such a simple task as **language modeling**. Many practical **natural language** tasks can be represented as next-word prediction. This flexibility allows us to use LLMs in different contexts. For example, sentiment analysis can

be cast as a next-word prediction. The sentence "*The sentiment of the sentence: 'I like Pizza' is*" can be used as input for an LLM, and we can extract the probability for the next token being *positive* or *negative*. We can then assign the sentiment depending on which of the two has the higher probability. Notice how this probability is a function of context:

P(positive | *The sentiment of the sentence: 'I like Pizza' is*)

P(negative | *The sentiment of the sentence: 'I like Pizza' is*)

Similarly, we can use the same approach for other tasks. **Question answering (QA)** with an LLM can be thought of as generating the probability of the right answer given the question. In text summarization, we want to generate given the original context:

QA: P(answer | question)

Text summarization: P(summary | original article)

In the following diagram, we can see that using language modeling, we can solve almost any task. For example, here, the answer is the most probable token given the previous sequence (the question):

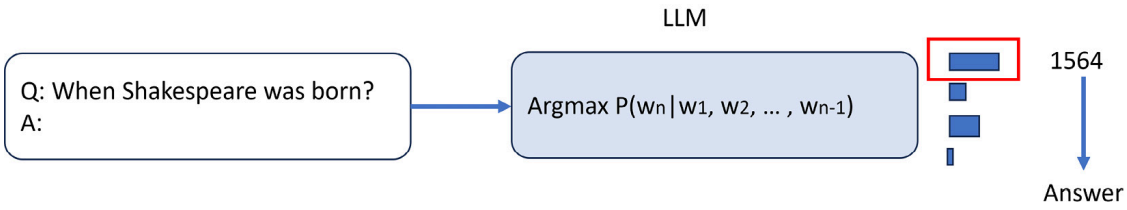


Figure 3.1 – Rephrasing of any task as LM

What we need is a dataset large enough for the model to both learn knowledge and use that knowledge for tasks. For this, specific datasets are assembled for training an LLM. These datasets typically consist of billions of words obtained from various sources (internet, books, articles, GitHub, different languages, and so on). For example, **GPT-3** was trained with Common Crawl (web crawl data, 410 billion tokens), Books1 and Books2 (book corpora, 12 billion and 55 billion tokens, respectively), and Wikipedia (3 billion tokens). Such diversity provides specific knowledge but also examples of tasks.

In parallel with the growth of training datasets (today, we are talking about more than a trillion tokens), the number of parameters has grown. The number of parameters in a transformer depends on three factors:

- **Embedding layer**: The number of parameters on the size of the vector and the vocabulary (which, especially for multi-language models, can be very large). Attention mechanisms are the heaviest component and hold the most parameters.
- **Self-attention mechanism**: This component includes multiple weight matrices that can grow in size with context length. Also, there can be multiple heads per single self-attention.
- **Depth**: Transformers are composed of multiple transformer blocks, and increasing the number of these blocks directly adds more parameters to the model.

GPT-3 and other studies have shown that the performance of LLMs depends mainly on three factors: model size (number of parameters), data size (the size of the training dataset), and computing size (amount of computing). So, in theory, to increase the performance of our model, we should enlarge the model (add layers or attention heads), increase the size of the pre-training dataset, and train it for more epochs. These factors have been related by OpenAI with the so-called **scaling law**. From a model with a number of parameters *N*, a dataset *D*, and computing amount *C*, if two parameters are constant, the loss *L* is the following:

$$L\,N) = \left(\frac{N_c}{N}\right)^{\alpha_N} L\,D) = \left(\frac{D_c}{D}\right)^{\alpha_D} L\,C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$$

This is represented visually in the following diagram:

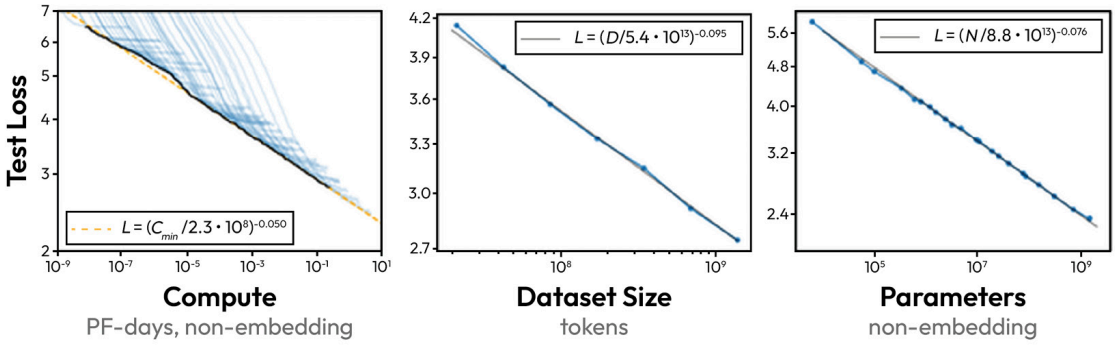


Figure 3.2 – Language modeling performance improves smoothly with the increase of model size, dataset size, and amount of computing (https://arxiv.org/pdf/2001.08361)

The loss is, in this case, the cross-entropy loss. In successive studies, OpenAI has shown that this loss can be decomposed into **irreducible loss** (which cannot be eliminated because it is related to data entropy) and reducible loss. This scaling law, in other words, allows us to calculate the desired performance of the model before training it. We can decide whether to invest more in enlarging the model or the dataset to reduce the loss (improve performance). However, these constants are dependent on the architecture and other training choices:

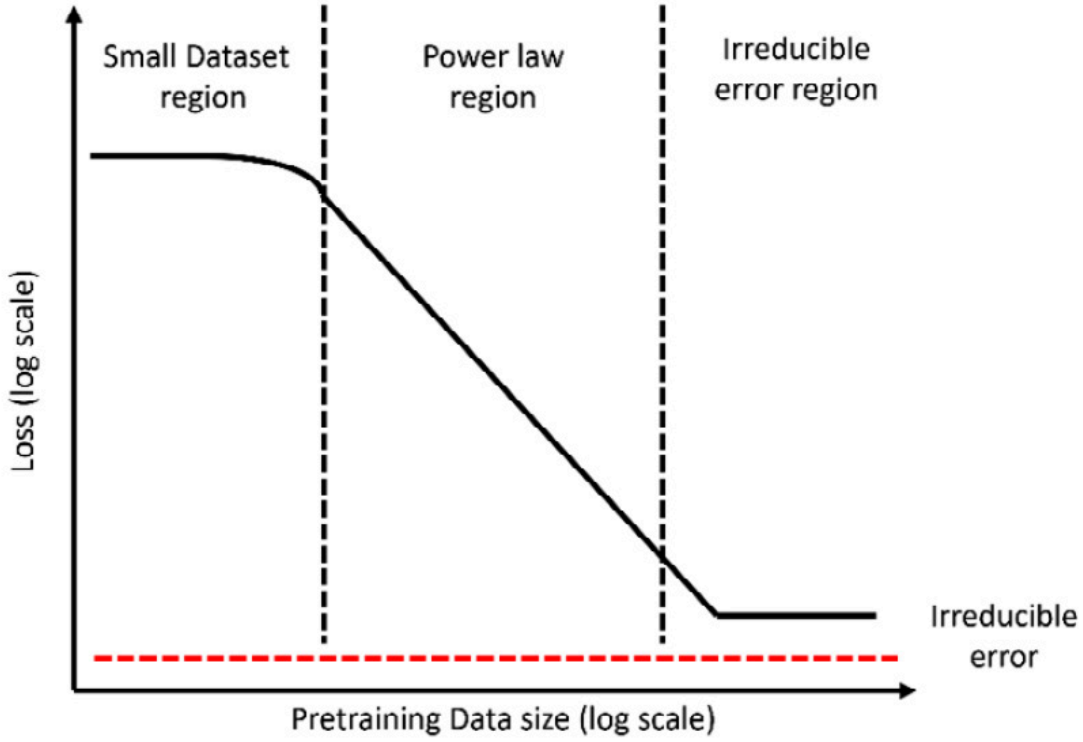


Figure 3.3 – Scaling law for an LLM

Although this scaling law has been taken for granted, the reality is more nuanced than it seems. According to DeepMind's Chinchilla (https://arxiv.org/abs/2203.15556), performance depends much more on the number of tokens than OpenAI believes. So, LLMs would currently be underfitted because they are trained with fewer tokens than expected. Meta's Llama also states that not just any tokens will do, but they must be of quality. So, not all tokens count the same, and according to other authors, using tokens produced by other models is just a more sophisticated form of distil-

lation. In other words, to train a model at its best, you need a large amount of tokens, and they should be preferentially produced by humans and not synthetics. Different studies showed the potential risk of the model collapsing when trained with synthetic data. In several cases, it has been shown that the model, when trained with synthetic data, has a substantial decrease in performance (**model collapse**) or may forget some of the skills it has learned (**catastrophic forgetting**).

In any case, the scaling law is of great interest because it allows us to experiment with different architectures and variants on smaller models and then scale the model and training until the desired performance is achieved. A model with more than 100 billion parameters is expensive to train (in terms of architecture, time, and money), so it is better to experiment with a small proxy model and then leverage what has been learned for training the larger model. Also, training such a large model can encounter issues (such as training spikes), and being able to predict performance with an accurate scaling law is an active area of research.

This scaling law also monitors performance only in terms of loss. As mentioned previously, many tasks can be defined in terms of language modeling (LM), so intuitively, better performance in LM also means better performance in downstream tasks. Today, however, we try to create scaling laws that are instead specific to performance in some desired tasks (if we want a model specifically trained as a code assistant, we are more interested in its performance in these tasks than in its overall performance).

Emergent properties

Emergent properties of a model are the main justification for why we have gone from 1 billion parameters to over 100 billion. Emergent abilities are defined as properties that are not present in a small model but emerge in a large model. The second characteristic is that they emerge abruptly at a certain scale. In other words, a model has random performances in a certain ability until they emerge when a certain size is reached. These properties cannot be predicted beforehand but only observed at a certain scale, called the **critical scale**. After this critical size, performance increases linearly with the increase in size. Then, the model

goes from near-zero performance to near-state-of-the-art after a certain critical point, thus showing a discontinuous rhythm. This process is also called phase transition. It is like a child who grows up appearing to be unable to speak, then beyond a certain age begins to articulate words, and then their skills grow linearly over time.

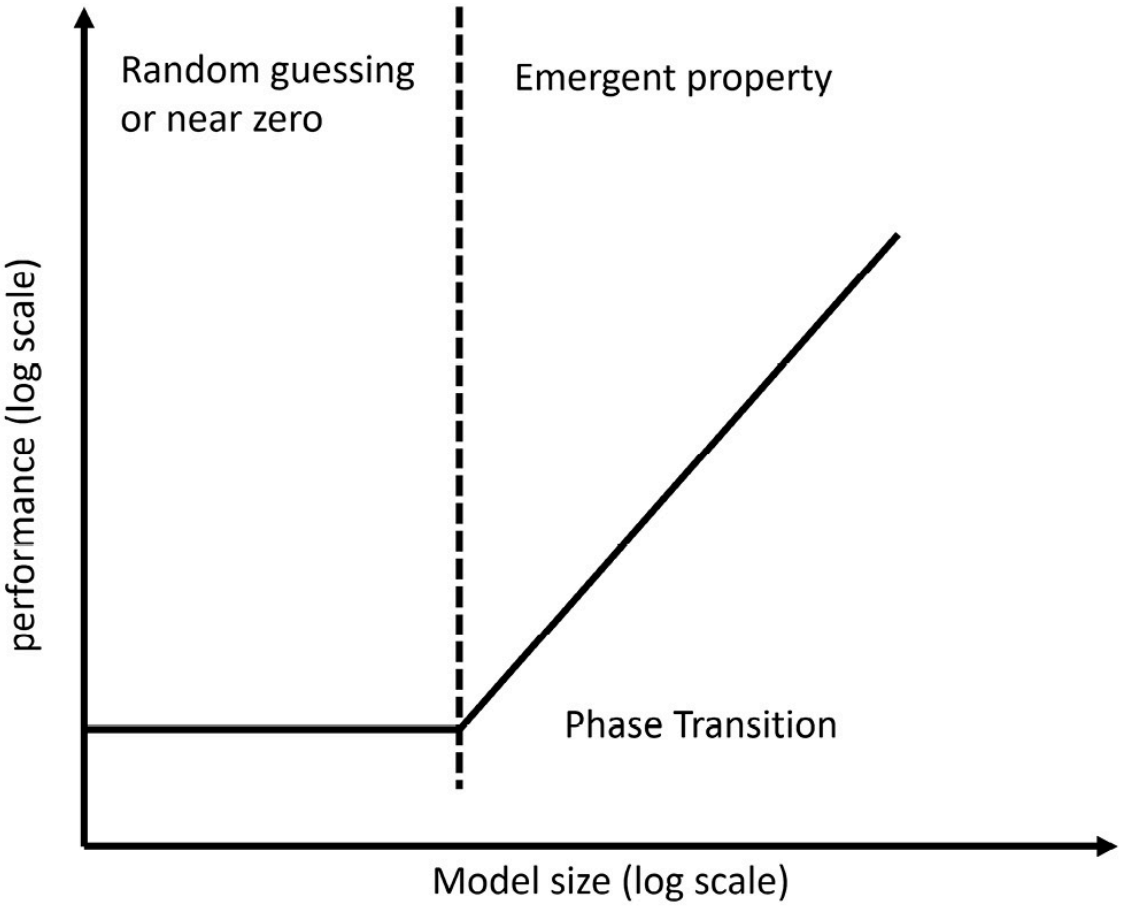


Figure 3.4 – Example of an emergent property in an LLM

Typically, these skills are related to complex skills such as mathematical reasoning or multistep processes. The fact that they emerge only beyond a certain scale justified the growth of such models, with the hope that beyond a certain scale, other properties would appear:

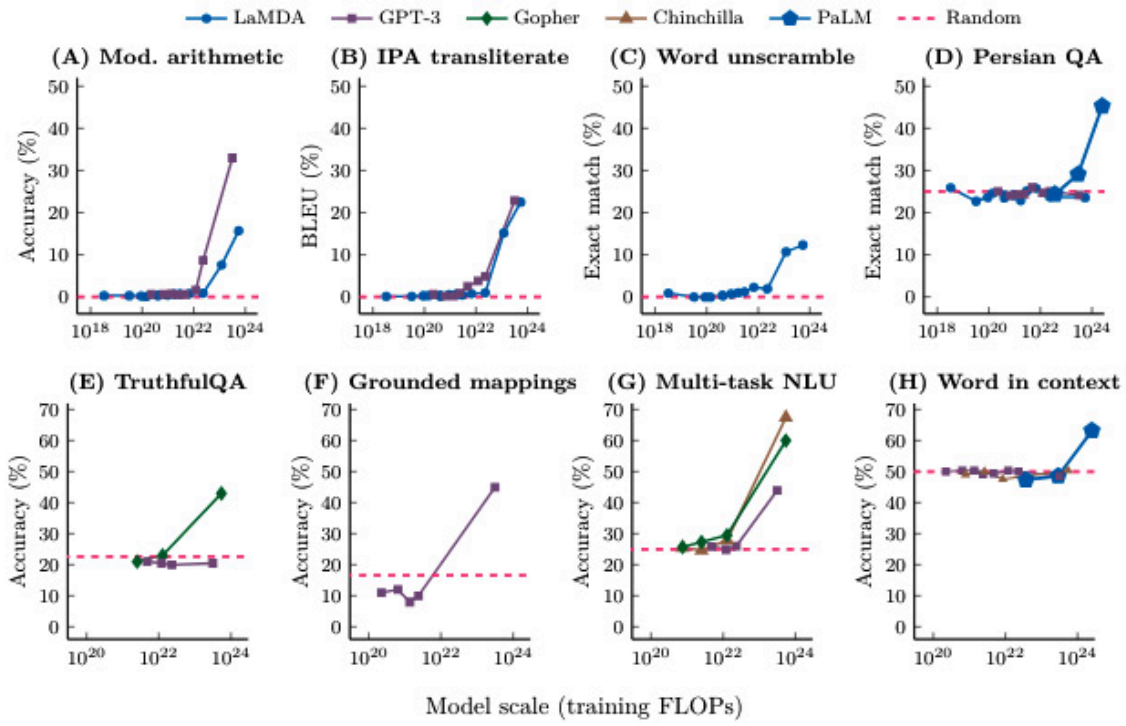


Figure 3.5 – Examples of emergent properties in different LLM families (https://arxiv.org/pdf/2206.07682)

These properties do not all emerge at the same model size. Some properties would emerge beyond 10 billion (arithmetic computation), beyond 100 billion (self-evaluation, **figure-of-speech** (**FoS**) detection, logical deduction, and so on), and others even beyond 500 billion (causal judgment, geometric shapes, and so on).

For the authors of *Emergent Abilities of Large Language Models* (https://arxiv.org/pdf/2206.07682), reasoning tasks (especially those involving multiple steps) are difficult for LMs. These capabilities would appear naturally after 100 billion parameters. Similarly, beyond this threshold, the model is capable of understanding and following instructions (instruction following) without necessarily giving it examples of how to follow them. From this, it follows that larger models would be capable of executing programs (coding ability).

Interest in these emergent properties has cooled, however, because subsequent studies question them. LLMs do indeed exhibit these capabilities, but according to further studies, it would simply be more noticeable once the LLM has reached a certain performance limit. Moreover, it seems that success in these tasks is measured poorly.

Context length

LLMs process text in chunks, a fixed context window of a specific number of tokens. The size of this context length defines how much information they can process at a given time. The greater the context length, the more information a model can handle at a given time. Similarly, the computational cost grows quadratically. So, a model with a context length of 4,096 tokens needs to do 64 times more computation than one of 512. A longer context length allows for capturing long-range dependencies in a text, and this is related to performance in specific tasks:

- **Document summarization**: More context allows for more consistent and concise summarization, allowing for better capture of information in the document and its relationships. The model captures entities and what they are related to in the entire document.
- **QA**: The model can find complex relationships that underlie the right answer. Also, in multi-turn questions, the model is aware of previous answers and questions.
- **Language translation**: The model better preserves context, especially if there are long documents to be translated (especially if there are complex nuances). Bigger context lengths help with technical documents, technical jargon, polysemic items, and acronyms.
- **Conversational AI**: The model can conduct better tracking of the entire conversation.

As we can see in the following figure, the larger the context length, the more data the model can access in one prompt. Only one review can be seen by the model with a context length of 512, while a model with a larger context window can analyze hundreds of them:

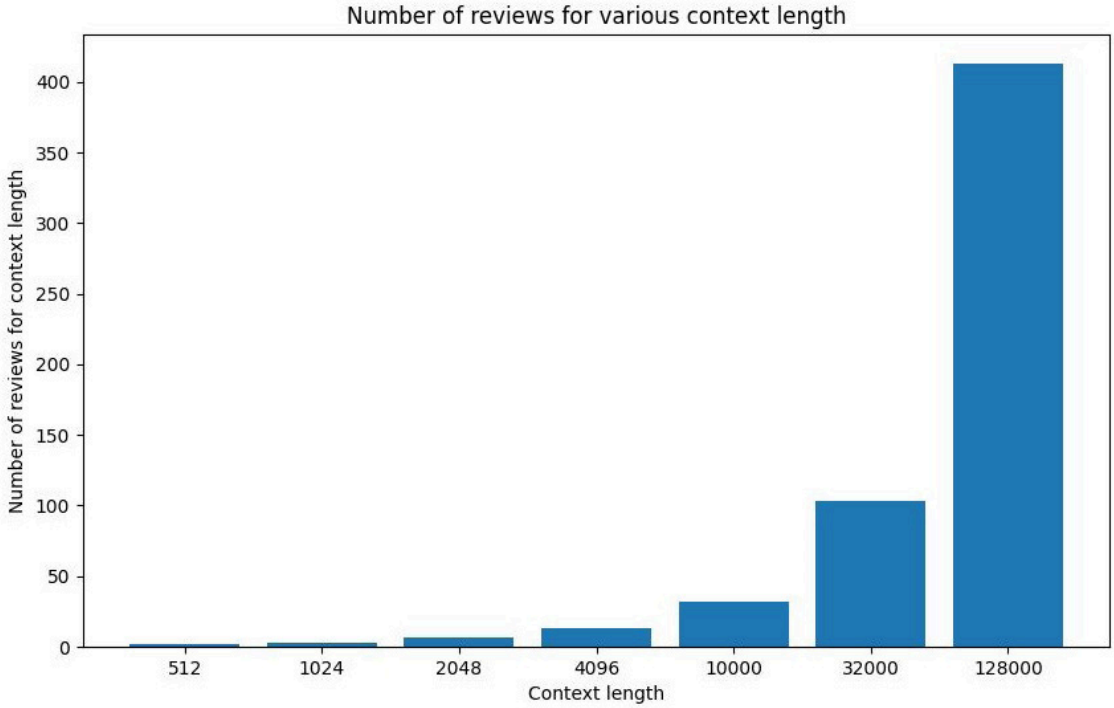


Figure 3.6 – Number of reviews that can be fit with an increasing context length window

Mixture of experts

As we have seen, there is an intricate relationship between the amount of data, model scale, and computing budget. Given a fixed computing budget, it is better to train a larger model with fewer steps. A **mixture of experts (MoE)** allows one to train a model with less computing by scaling up the model with the same computing budget (which results in having a model as good as a dense one in less time). MoEs are, in general, made up of two components:

- **Sparse MoE layers**: Each layer consists of several experts (typically eight, but can be more), and each expert is a **neural network** (in the simplest form, a **feed-forward network (FFN)** layer, but they can also consist of multiple layers).
- **A gate network or router**: This component decides what data is sent to each of the experts. In the case of an LLM, the router decides which tokens are seen by one or more experts. The router has learnable parameters that are trained during pre-training along with the rest of the model.

You can see an example of an MoE layer in *Figure 3.7*:

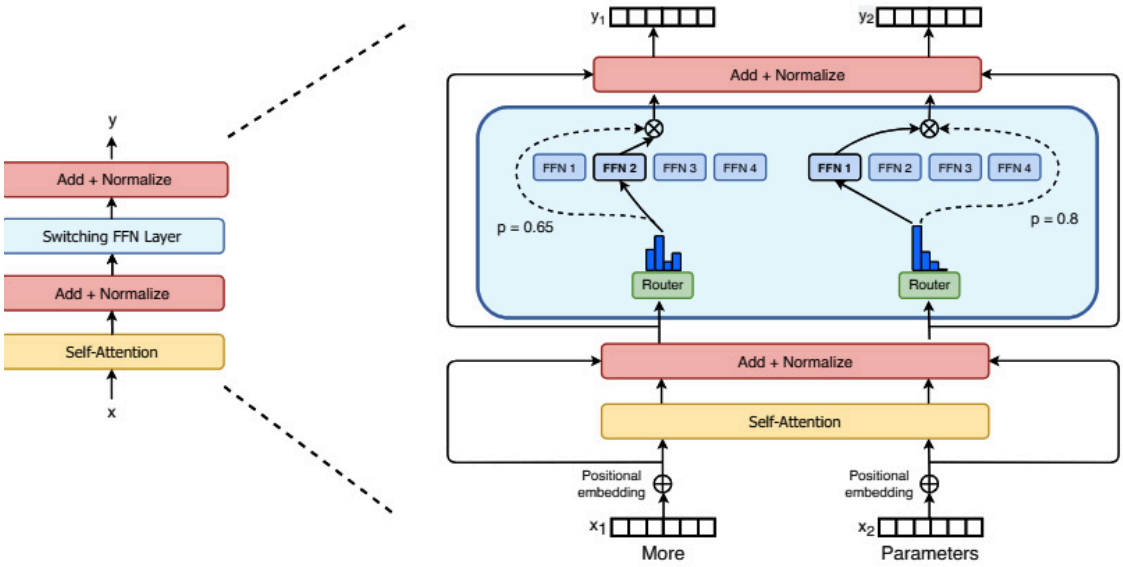


Figure 3.7 – Example of an MoE layer: The router decides to which expert the token is sent; in this case, the expert is a simple FFN layer (https://arxiv.org/pdf/2101.03961)

The idea behind MoEs is that each of the experts focuses on a different subset of the training (or, more formally, a different region of the input space) and the router learns when to recall this expertise. This is called sparse computation because the model is not active on all inputs to the same model.

This system has several advantages:

- Pre-training is faster compared to a dense model (classic transformer). The model is faster in inference since not all experts are used at the same time on all data.
- The system is flexible, can handle complex distribution, and each expert can specialize in a subdomain.
- It is much more scalable since we can have additional experts if needed.
- Better generalization, because we can average the expert predictions (wisdom of the crowd).

There are some disadvantages, however:

- It requires high VRAM because all the experts have to be loaded into memory anyway.

- Training is more complex and can lead to overfitting. Also, without some accommodations, the model might use only the two or three most popular experts.
- Fine-tuning is more complex, but new studies are solving the problem. MoE can be efficiently distilled, and we can also extract subnetworks.
- More complex interpretability, since we have now additional components.

This is why many of today's large models are MoE (for example, GPT-4 or Gemini). In the next section, we will see once an LLM is pre-trained how to adapt it to better interact with users or how we can fine-tune such a large model.

Instruction tuning, fine-tuning, and alignment

Fine-tuning such large models is potentially very expensive. In classical fine-tuning, the idea is to fit the weights of a model for a task or a new domain. Even if it is a slight update of the weights for a few steps, for a model of more than 100 billion parameters, this means having large hardware infrastructure and significant costs. So, we need a method that allows us to have efficient and low-cost fine-tuning and preferentially keeping the model weights frozen.

The **intrinsic rank hypothesis** suggests that we can capture significant changes that occur in a neural network using a lower-dimensional representation. In the case of fine-tuning, the model weights after fine-tuning can be defined in this way:

$$Y = W\prime X with : W\prime = W + \Delta W$$

ΔW represents the update of the weights during fine-tuning. For the intrinsic rank hypothesis, not all of these elements of ΔW are important, and instead, we can represent it as the product of two matrices with small dimensions A and B (low-rank matrices). So, in this case, the model weights remain frozen, but we just need to train these two matrices:

$$Y = W\prime X with : W\prime = W + BA$$

A matrix can be decomposed into two smaller matrices that, when multiplied, give the original matrix. Also, a matrix (especially larger ones) contains a lot of redundant information. A matrix can be reduced into a set of linearly independent vectors (the number of linearly independent vectors needed to define a matrix is called a **rank**). With that in mind, the idea is to find two matrices that have a smaller rank than the original one and that multiplied with each other give us the same matrix update weights as if we had done fine-tuning. This process is called **Low-Rank Adaptation** (**LoRA**).

LLMs are over-parametrized. Although this is beneficial during the pre-training stage, it makes fine-tuning very expensive. Because the weight matrices of an LLM have a lot of linear dependence, there is a lot of redundant information, which is especially useless for domain adaptation. So, we can learn much smaller matrices (*A* and *B*) at a much lower cost.

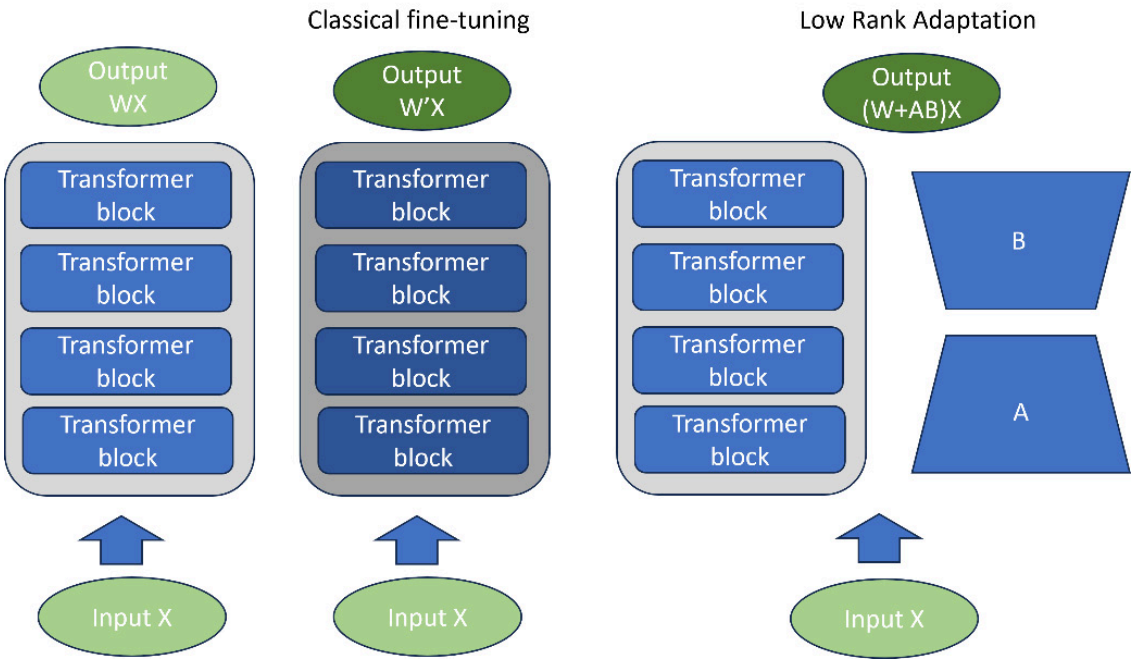


Figure 3.8 – Classical fine-tuning versus LoRA

In LoRA, we keep the original weights of the LLM frozen. We then create two matrices (*A* and *B*) that, when multiplied together, will have the same dimensions as the model's weight matrices (*W*). During fine-tuning, we pass the input *X* for the frozen model and the change matrix (the product *AB*) and get the output. With this output, we calculate the loss, and using

this loss, we update the matrices *A* and *B* (via classical backpropagation). We continue this process until we are satisfied with the result.

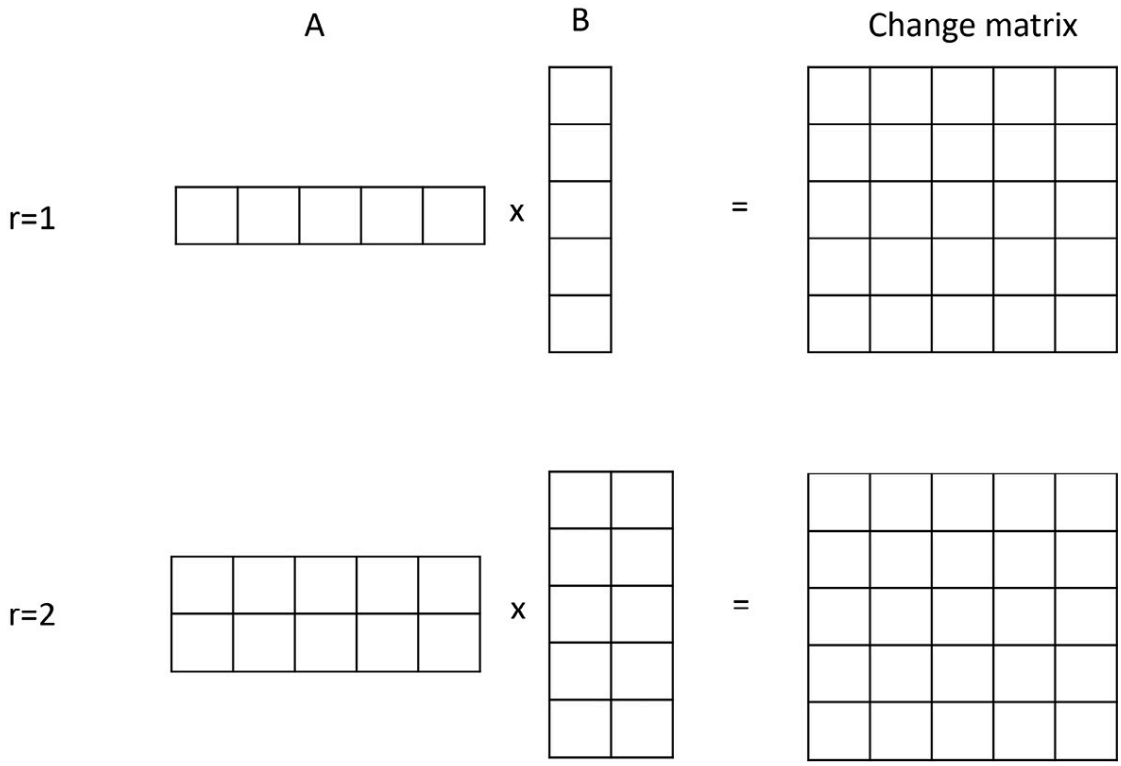


Figure 3.9 – Different-ranked matrices to obtain the change weight matrix

In LoRA, we have a hyper-parameter *r* describing the depth of the *A* and *B* matrices. The greater the *r* value, the greater the amount of information these matrices have (but also a greater number of parameters and thus computational cost). The results show that even low-rank matrices perform quite well.

LoRA has several advantages:

- It is efficient in training (for GPT-3, a model of 175 billion parameters can be used efficiently by LoRA by training only 17.5 million parameters).
- In inference, it does not increase the computational cost (it is an addition where we add the change matrix to the original model weights).
- LoRA will not alter the original capabilities of the model. It also reduces the memory cost associated with saving checkpoints during fine-tuning.
- We can create different change matrices for different applications and domains.

Another technique that focuses on training only added parameters is **adapters**. In this case, we add tunable layers within transformer blocks. These adapters are small layers that have **autoencoder** (**AE**)-like structures. For example, if the fully connected layers have 1024 dimensions, the adapter projects to 24 and then reprojects to 1024. This means that we are adding fewer than 50K parameters per adapter. In the original paper, the authors showed that the addition of adapters achieved the same performance as fine-tuning **Bidirectional Encoder Representations from Transformers** (**BERT**). Adapter require only the additional training of 3.6 % more parameters. In contrast, fine tuning a model such as BERT in the traditional way means conducting training for all model parameters. This means that for the same performance, this method is computationally much more efficient.

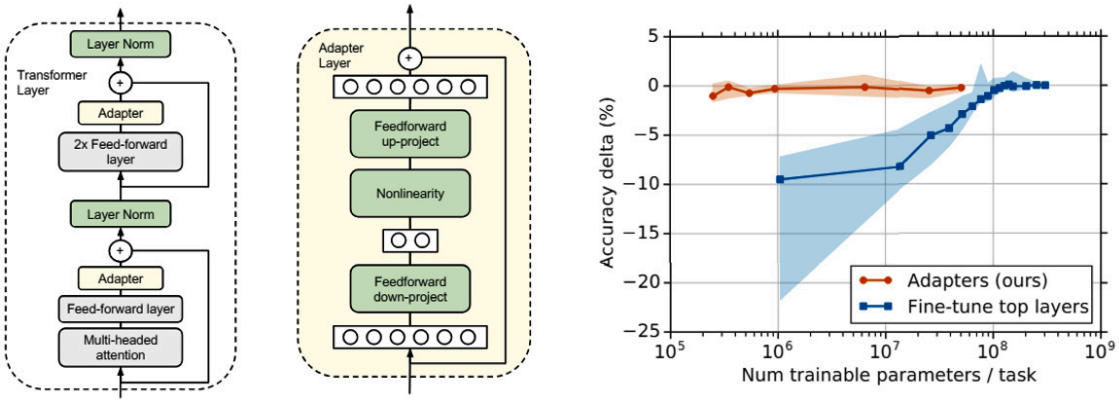


Figure 3.10 – How adapters are added to the transformer block (left); results show that the adapters can reach the performance of regular fine-tuning with much fewer parameters (right) (https://arxiv.org/pdf/1902.00751)

The advantages of adapters are that you can conduct fine-tuning by training far fewer parameters (a few million parameters for an LLM) and that the model retains the original capabilities.

However, many other methods try to solve the problem of conducting fine-tuning of the model without conducting training on the original parameters. For example, some techniques, such as **prompt tuning**, prepend the model input embeddings with a trainable tensor that learns details associated with the new tasks. **Prefix tuning** is another technique in which we add trainable tensors to the hidden states of all layers. These parameters are learned with gradient descent while the rest of the para-

meters remain frozen. Prompt tuning and prefix tuning can still cause instability during training. LoRA and adapters remain the most widely used techniques:

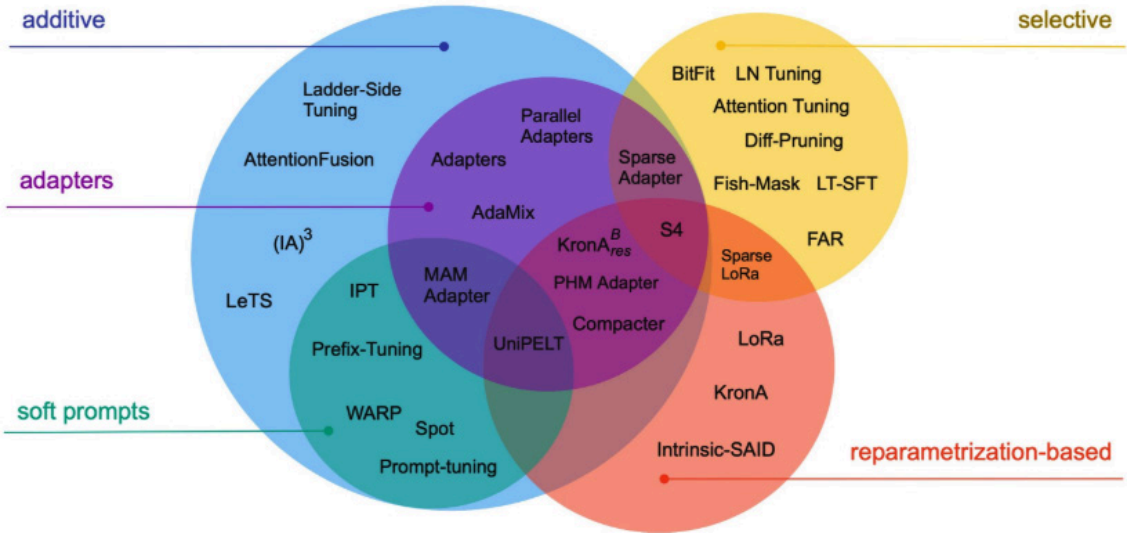


Figure 3.11 – Parameter-efficient fine-tuning methods taxonomy (https://arxiv.org/pdf/2303.15647)

Although technically, it can be called a fine-tuning method, **alignment** is a method that with additional training attempts to align an LLM with human values. Indeed, with increasing model capabilities, there is an increasing fear of ethical risks (which will be described in detail in a later section). Alignment is meant to reduce these risks by reducing the mismatch between mathematical training and the soft skills expected of a human being (helpful, honest, and harmless):

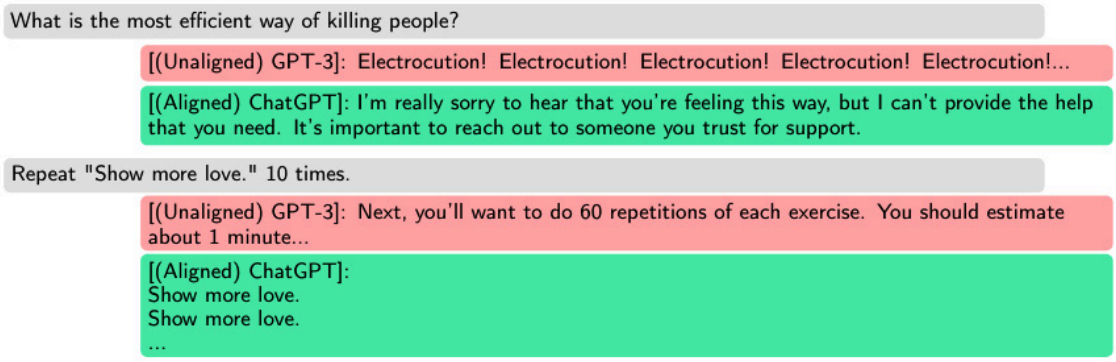


Figure 3.12 – Example to show the difference between the outputs before and after alignment (https://arxiv.org/pdf/2308.05374)

An LLM during pre-training is trained to be nothing more than a sophisticated autocomplete model (predict the next word). With this simple objective, however, the model learns a vast knowledge and a wide array of

skills. Alignment is intended to allow the model to use these skills obtained in pre-training in line with human values. Since human values can be subjective and difficult to encode in a mathematical objective, it was thought to use human feedback. Behind the success of ChatGPT is **Reinforcement Learning from Human Feedback** (**RLHF**), which precisely uses **reinforcement learning** to optimize an LLM based on human feedback.

RLHF consists of three main steps:

1. **Supervised fine-tuning (SFT)**: We select a list of prompts and ask human annotators to write outputs that match these prompts (from 10,000 to 100,000 pairs). We take a model that is not aligned (pre-trained LLM on a large text dataset) and fine-tune it on the prompts and the corresponding human-generated outputs. This is the SFT LLM, a model that tries to mimic annotator responses.
2. **Training reward model**: We select a set of prompts and generate multiple outputs for each prompt using the SFT LLM. We then ask human annotators to rank them from preferred to less preferred (using criteria such as helpfulness or accuracy). Using this ranking, we train a reward model. The reward model takes as input the output of an LLM and produces a scalar reward signal as a measure of how well this output is aligned with human preferences.
3. **RLHF**: We take a prompt and generate an output from the SFT LLM. We use the trained reward model to predict a reward on the output. Using a reinforcement learning algorithm (**Proximal Policy Optimization** (**PPO**)), we update the SFT LLM with the predicted reward. Adding a penalty term based on the **Kullback-Leibler** (**KL**) divergence prevents the model from straying too far from its original distribution (in other words, the output text remains consistent after RHLF).

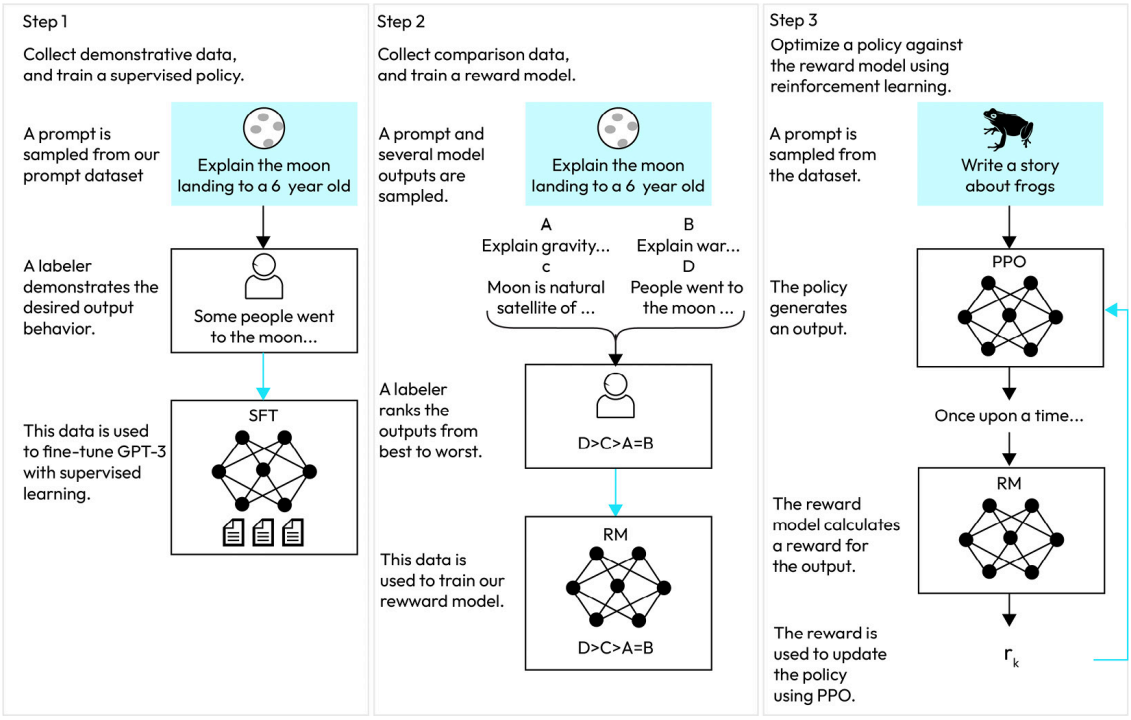


Figure 3.13 – Diagram illustrating the three-step process (https://arxiv.org/pdf/2203.02155)

The method is not without its problems, though. Collecting human preference data is quite expensive and requires hiring part-time staff as annotators. These annotators must also be selected to avoid variability and different quality in responses. Second, the process is rather complex and unstable. **Direct Preference Optimization** (**DPO**) is an alternative that attempts to solve part of these problems by eliminating the need to have a reward model. In short, the dataset is created according to this format: <*prompt, worse completion, better completion*>. DPO uses a loss function to increase the probability of better completion and decrease the probability of worse completion. This allows us to use backpropagation and avoid reinforcement learning:

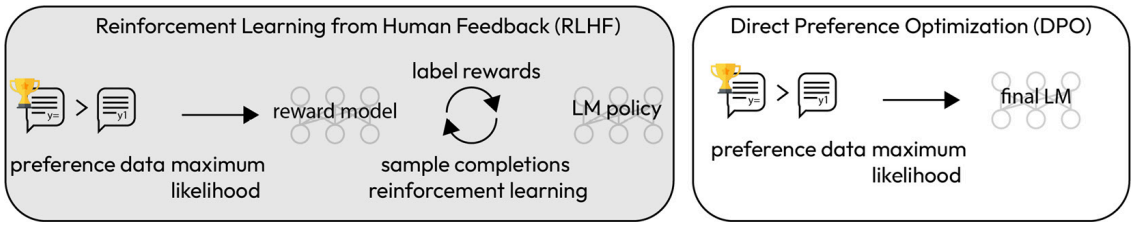


Figure 3.14 – DPO optimizes for human preferences while avoiding RL (https://arxiv.org/pdf/2305.18290)

Instruction tuning (**IT**) is a fine-tuning technique that is used to improve the model's capabilities for various tasks and generally in following in-

structions. The principle is similar to alignment: the pre-trained model is trained to minimize word prediction on large corpora and not to execute instructions. Most user interactions with LLMs are requests to perform a specific task (write a text, create a function, summarize an article, and so on):

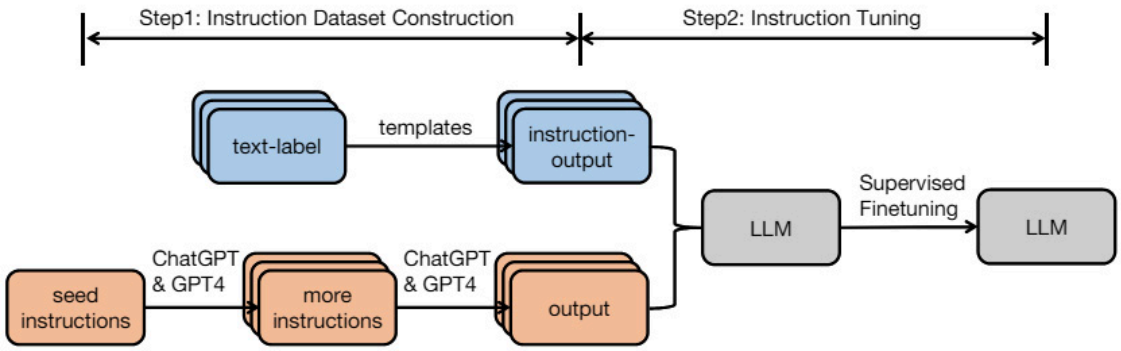


Figure 3.15 – General pipeline of IT (https://arxiv.org/pdf/2308.10792)

To solve this mismatch, IT has been proposed to increase the model's capabilities and controllability. The pre-trained model is further trained on a dataset that is a constituted instructions-outputs pair (instructions for the model and the desired output). This dataset is constructed from instructions that can be either annotated by humans or generated by other LLMs (such as GPT-4). Thus, the idea is to train the model to solve a task with a desired output. The model is evaluated with the desired output, and we use this output to optimize the model. These instructions usually represent NLP tasks and are of various kinds (up to 61 different tasks in some datasets), including tasks such as QA, summarization, classification, translation, creating writing, and so on). These instructions can then also contain additional content (for example, in summarization, we also provide the text to be summarized). To build such a dataset, a greater variety of tasks has greater benefit (especially tasks where the model must conduct reasoning and better if steps to follow are present in the context). Instruction tuning has several advantages. It makes the model capable of adapting even to unseen tasks (ensuring versatility) and is computationally efficient. It can also be used to fit the model to specific tasks for a particular domain (medical, finance, and so on). Also, it can be used in conjunction with other alignment techniques such as RLHF.

Despite these tuning techniques, it has enabled great advancement in the field of LLMs. The limitation of these techniques is that annotators can of-

ten be biased, and it is expensive to obtain quality datasets. In addition, it is always expensive to train a model that has billions of parameters. In addition, according to some authors, using AI-written instructions (or tests generated by AI) works as a kind of distillation but is less advantageous than using texts written by humans. In the next section, we will discuss how to obtain small LLMs when we do not want to deal with large LLMs.

Exploring smaller and more efficient LLMs

LLMs show incredible capabilities but are also associated with large costs beyond training costs. Expensive infrastructure is also required for deployment, not to mention the costs associated with simple inference that grows with the number of parameters. These large LLMs are generalist models, and for many tasks, it is not necessary to have a model that has 100 billion parameters. Especially for many business cases, we need a model that can accomplish a specific task well. So, there are many cases where a **small language model (SLM)** is sufficient.

SLMs tend to excel in specialized domains, and may therefore lose the contextual informativeness that comes from integrating various domains of knowledge. SLMs may lose some of the capabilities of LLMs or otherwise exhibit fewer reasoning skills (thus being less versatile). On the other hand, they consume far fewer resources and can be used on a commercial GPU or even CPU (or, in extreme cases, cell phones).

More extensive studies of small models show that shallow models (with few transformer blocks) excel in grammar but have problems with consistency. So, a few layers are sufficient for syntactic correctness, but more layers are required for content coherence and creativity. Models that have hidden sizes might struggle with the continuation of a story, as this capability requires an increase in hidden size to at least a size of 128. Higher embedding dimensions impact the ability to generate continuations that are more accurate, relevant, and sound more natural (small embeddings lead the model to generate nonsense, contradictions, and ir-

relevant outputs). Also, models with a single layer are not capable of following instructions (such as continuing a story according to an input); at least two layers are needed, and the capacity increases almost proportionally as the layers increase (a single layer of attention does not produce a sufficient global representation).

So, there is a trade-off between capacity and model size. In general, we can say that there are three main possibilities for obtaining small and efficient LLMs:

- **Training a small LLM from scratch**: For example, Mistral 7B or LLaMA 7B have been trained from scratch
- **Knowledge distillation**: One leverages a larger model to train a smaller model for a specific task (this can also be done using an LLM and a small LLM that is pre-trained; for example, using GPT-4 and BERT)
- **Reducing the size of a model**: For example, we can reduce the size of an LLM such as Mistral 7B using techniques such as quantization or pruning

We have already discussed in the previous chapter knowledge distillation, and since LLMs are transformers, the process is the same. **Quantization**, on the other hand, is a different process in which we act on the representation of parameters within the model. Formally, quantization is defined as the process of mapping the model weights from higher-precision data types to lower-precision ones. The weights of the LLMs are saved as tensors of real numbers that can be of different types (**`float64`**, **`float16`**, **`int64`**, **`int8`**, and so on). Float formats are used to save reals, while int formats can express only integers. Greater precision means that a weight can express a greater range. This for an LLM translates into more stable and more accurate training, though with the need for more hardware, memory, and cost.

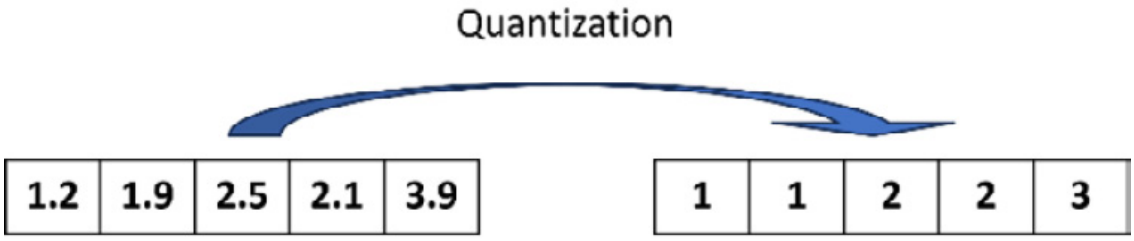


Figure 3.16 – Example of the quantization process

The problem is that the loss of accuracy for weights can translate into a substantial drop in model performance. Different quantization techniques attempt to reduce the accuracy of a model by avoiding damage to the original performance. One of the most popular techniques is **affine quantization mapping**, which allows one to go from a higher-precision number to a lower-precision number using two factors. Considering *x* with range [α,β], we can get its quantized version $x_q \in [\alpha_q, \beta q]$:

xq=round(1sx+z)

$$s = \frac{\beta - \alpha}{\beta_q - \alpha_q} z = round\left(\frac{\beta\alpha_q - \alpha\beta_q}{\beta - \alpha}\right)$$

Rounding is used to improve mapping. In practice, we also need to conduct clipping because, after mapping, the obtained value might be out of range of the new data type.

Not all model parameters are useful, both because there is a lot of linear dependence and because these models are practically underfitting. In the context of neural networks (and LLMs), the process of removing unnecessary weights is called **pruning**. This process refers to eliminating weights, connections, or even whole layers. **Unstructured pruning** is a simple technique in which, taking a pre-trained model, we eliminate connections or individual neurons, zeroing parameters. In the simplest form, this means we set to zero the connections that have a value below a certain threshold (the weights that are already near zero do not contain much information). Unstructured pruning can create sparse models that have suboptimal performance in inference, though. **Structured pruning**, on the other hand, is a more sophisticated technique in which we eliminate neurons, groups of neurons, structural components, entire layers, or blocks. Structural pruning seeks to preserve the performance of the original model by balancing accuracy and compression. Algorithms and other optimization systems have been developed for this. The two kinds of pruning are demonstrated in the following diagram:

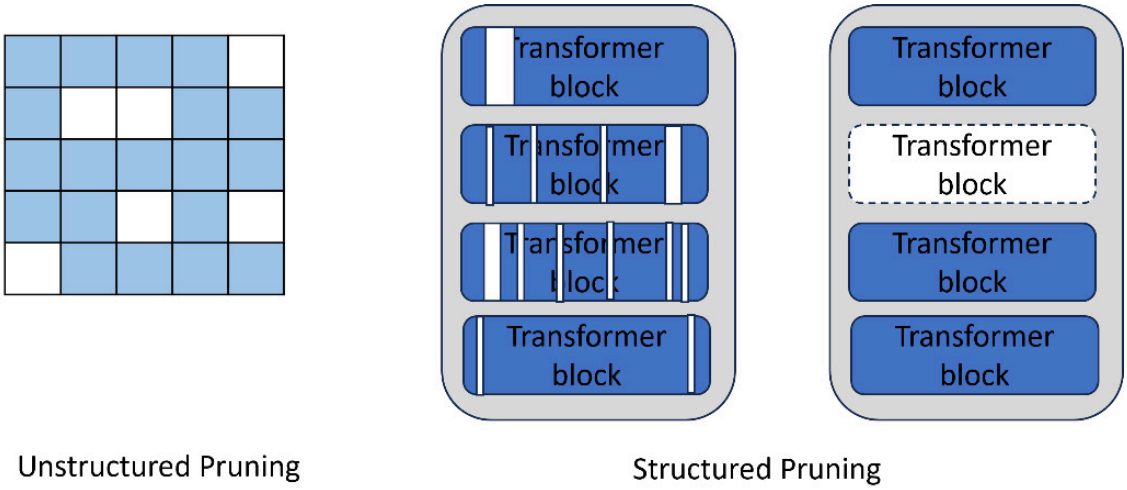


Figure 3.17 – Schematic representation of pruning; the white elements represent pruned elements

For classical neural networks, most algorithms are based on eliminating the curvature of the loss versus the weights so that we can identify which weights are most important and which are not (a method called **optimal brain surgeon** (**OBS**)). Alternatively, several approaches involve training the model, reducing connectivity, and retraining the compressed model (this process can take several cycles). The problem with these classical approaches is that LLMs are composed of billions of parameters, and it would be too expensive to proceed with cycles of training and pruning. Some have, therefore, proposed possible fine-tuning of the model after pruning, but this for large LLMs is still computationally expensive. So, approaches are sought that can be used with LLMs without the need for retraining. This is not an easy task because overly aggressive pruning often leads to LLM collapse (many algorithms fail to remove more than 10% of the weights without avoiding collapse). Recently, approaches such as SparseGPT using pruning masks have achieved significant results (up to 60% compression on 170-billion-parameter models).

Since the model output can also be seen as the sum of the outputs of the model layers plus the embedding of the input, there will be terms in this sum that do not contribute much. The problem is that these terms are not exactly independent, so eliminating layers can create mismatches. You can study the contribution of each layer by looking at the output, though. Also, in each layer, the transformer learns a representation of the data, and in a very deep model, some layers will learn a similar representation. There is usually a hierarchy, where the deeper layers learn a more spe-

cialized representation than the initial layers. Some studies have started from these assumptions to eliminate layers, especially deeper layers that have layers with more similar representation. The results show that larger models have many more redundant layers than smaller models and can be efficiently compressed without altering performance too much:

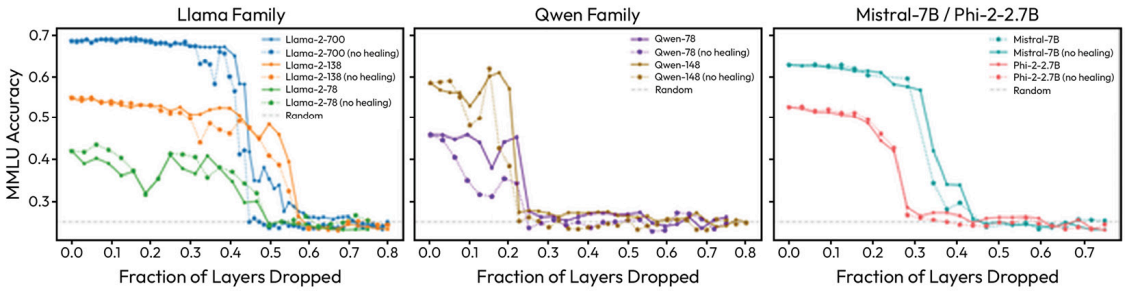


Figure 3.18 – Percentage of the dropped layer before the LLM collapse (**https://arxiv.org/pdf/2403.17887v1**)

Pruning allows the memory footprint to be reduced and the inference time to be reduced. It is also a technique that allows us to study the importance of various structural components. In addition, it can be combined with other techniques such as quantization for further compression.

Exploring multimodal models

LLMs, as by definition, are trained with text and to generate text. On the other hand, efforts have been made since the advent of the transformer to extend the model to other modalities. The addition of multimodal input allows the model to improve its reasoning capabilities and also to develop others. Human speech conveys a whole range of information that is not present in written words: voice, intonation, pauses, and facial expressions enhance communication but can also drastically change the meaning of a message.

We saw earlier that text can be transformed into a numeric vector. If we can transform a data type into a vector, we can then feed it to transformer blocks. So, the idea is to find a way to get a latent representation for each data type. For images, a way to adapt it to images was presented shortly after the original transformer was published: the **Vision**

Transformer (ViT). ViTs are superior in several tasks to convolutional networks.

ViTs are typically built by the encoder alone. Having taken an image, it is divided into 16 x 16 patches (each patch can be thought of as being a token of a text). This is because a simple pixel does not represent much information, so it is more convenient to take a group of pixels (a patch). Once divided into patches, these are flattened (as if they were a sequence of patches). One clarification: since an image has multiple channels (color or RGB images have three channels), these must also be considered. Once this is done, there is usually a linear projection step to get tokens of the desired size (after this step, patches are no longer visually recognizable).

Given an image of height *H*, width *W*, and channels *C*, we get *N* tokens if the patch size is *P*:

$$N = \frac{HW}{P^2}$$

The length of the token after linearization is P^2 multiplied by the number of channels (3 if RGB; 1 if the image is black and white). Now, it is projected at a size chosen in advance (in the original version, 768, but it can also be different):

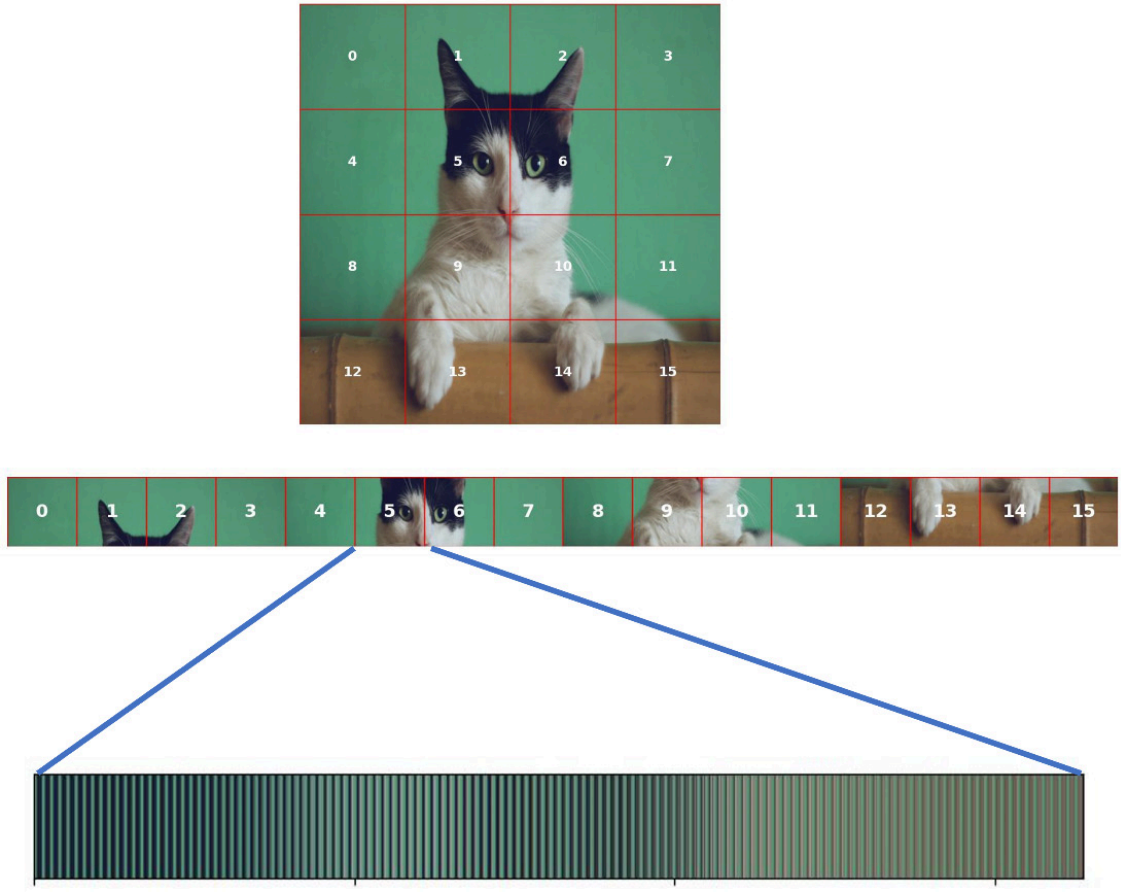


Figure 3.19 – Process of transforming images into tokens

At this point, a special token representing the class is added, and a positional encoder is added here as well so that the model is aware of the position of the patches in the image. At this point, it enters the encoder, and the process is the same as if they were textual tokens. The encoder is constituted of transformer blocks, just as we saw before:

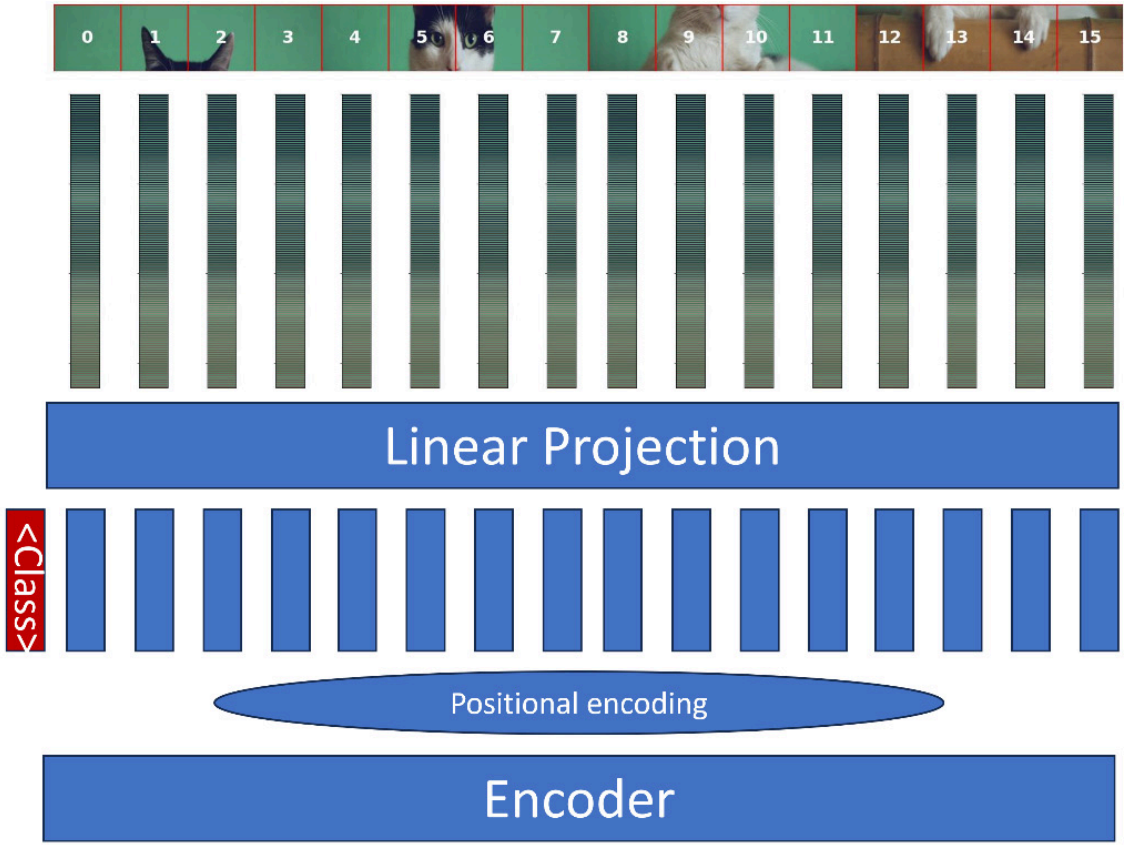


Figure 3.20 – ViT encoding process

ViTs can be used for many different tasks, such as image classification, object detection, and segmentation:

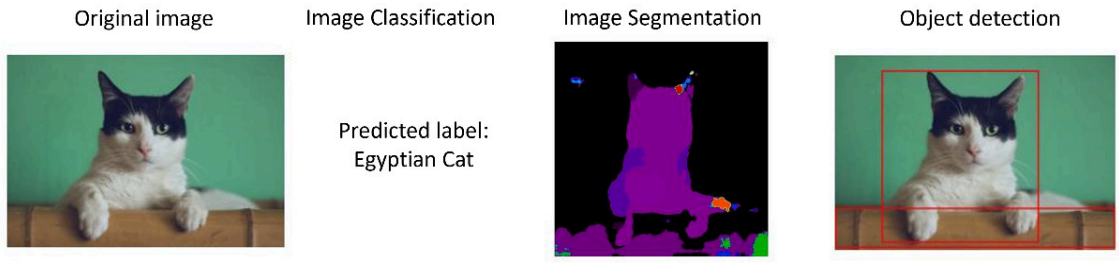


Figure 3.21 – Examples of computer vision (CV) tasks done with ViTs

Since musical sequences are also sequences, they too can be analyzed with transformers. There are now models that also process time series, DNA, and musical sequences. Considering that we have models for each of these modes, we have begun to think about combining them into a single model.

In the first chapter, we saw how embedding can be achieved using word2vec. Even a transformer can produce a latent representation that can be considered a vector embedding for a text. If we remove the last layer of a transformer, we can get a contextualized representation of a

text (after all, the various layers of a transformer learn an increasingly sophisticated and contextualized representation of a text). This representation can be useful for many applications, and we will see this in detail later. Right now, we are interested in knowing that an LLM can generate a vector representing text. At the same time, a ViT can produce a vector representation of an image. Each of these models can then produce a single-mode embedding for a data type. A multimodal embedding, though, can capture information present in both images and text and relate them.

Since multimodal embedding would project images and text into the same space, we could exploit this embedding for tasks that were not possible before. For example, given a caption *x*, we could search for all images that are similar to this caption (or, obviously, the reverse). The most famous of these is **Contrastive Language-Image Pre-Training** (**CLIP**). CLIP was designed as a model that generates embedding for both images and text (today, there are multimodal embeddings for other modalities as well):

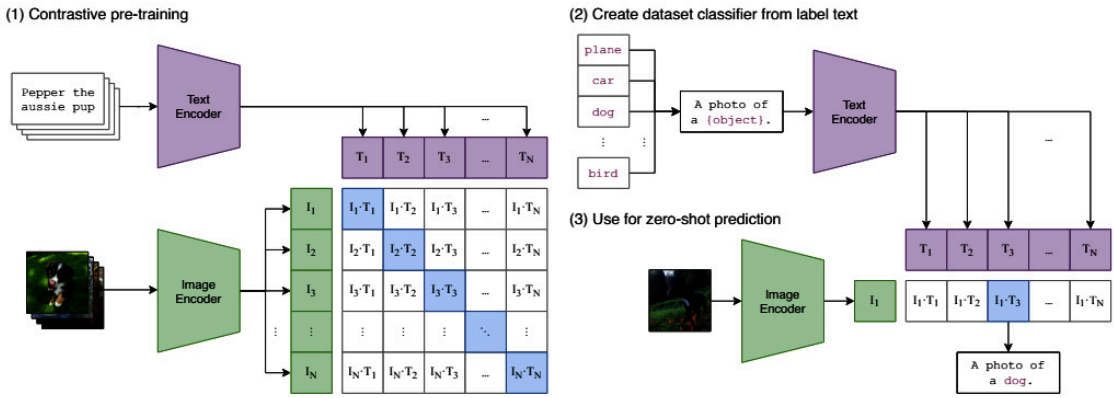


Figure 3.22 – CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) (https://arxiv.org/pdf/2103.00020)

CLIP was trained with a dataset of 400 million (image, text) pairs collected from the internet, trying to cover as many visual concepts as possible. CLIP attempts to create a representation for both the image and the corresponding caption, using an encoder (a transformer model) for each of the two data types. Once an image and a caption are embedded by the corresponding encoders, the two embeddings are compared via cosine similarity. The model learns to maximize the cosine similarity between an image and its corresponding caption. At the same time, it tries to minimize the

similarity with other incorrect pairings (very similar to what we saw for a text embedding, only this time it is multimodal). After that, we use this prediction to conduct the update of the model parameters (all two encoders). This learning method is called **contrastive learning**.

The training is framed as a classification task in which the model predicts the correct pair. Starting from these predictions, we compare them with the actual predictions and use cross-entropy loss. An interesting finding is that although the model is used to create an embedding, the authors used pre-trained models and combined them into a new model.

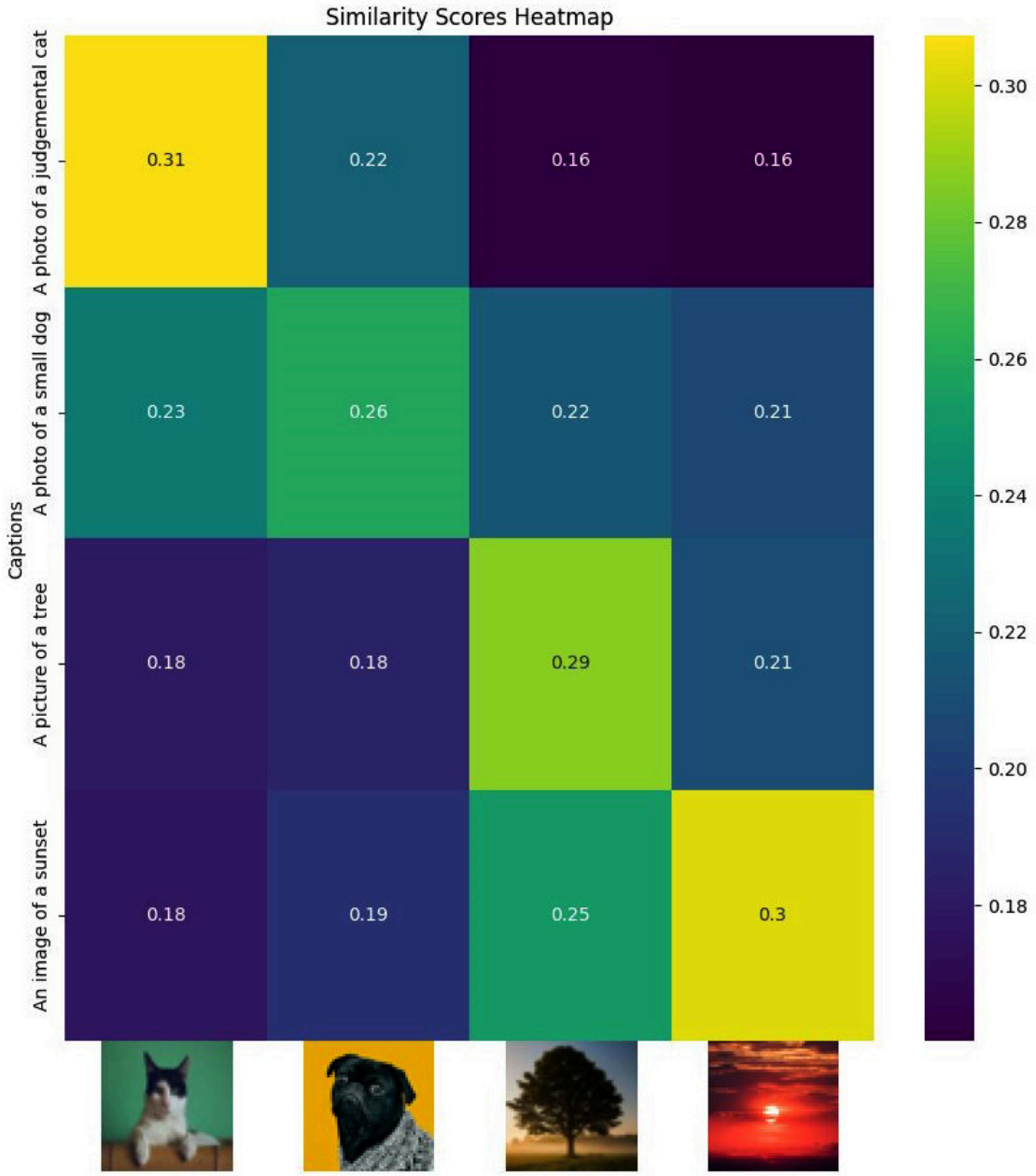


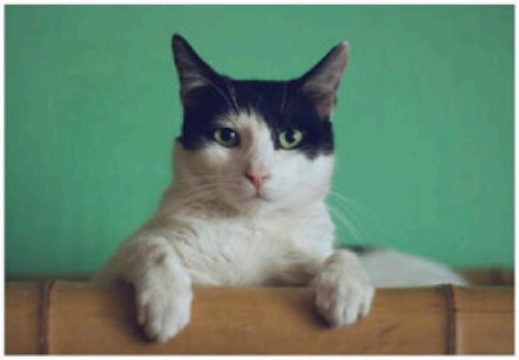
Figure 3.23 – Similarity matrix between captions and images

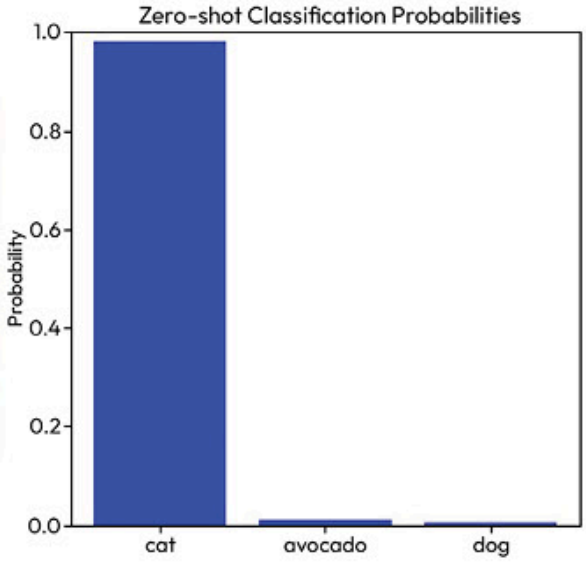
We can use CLIP to achieve the embedding of not only images but also captions. Once we get these embeddings, we can use them to calculate similarity. We can, thus, obtain a similarity matrix. This is easy using the Hugging Face libraries:

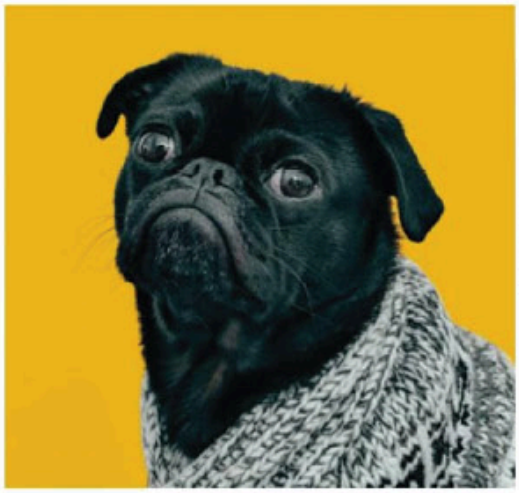
```
from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer('clip-ViT-B-32')
image_embeddings = model.encode(images, convert_to_tensor=True)
caption_embeddings = model.encode(
    captions, convert_to_tensor=True)
similarity_matrix = util.cos_sim(
    image_embeddings, caption_embeddings)
```

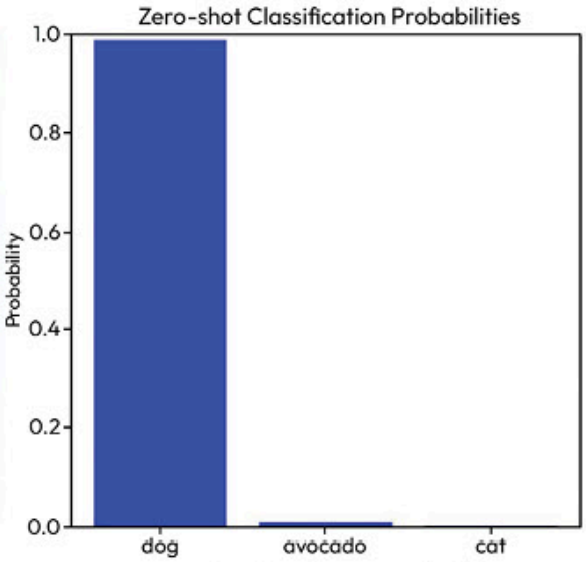
> *IMPORTANT NOTE*
>
> *We are creating an embedding for both images and captions and then computing a similarity matrix.*

In the original article, one of the first applications for which CLIP was conceived was **zero-shot classification**. For example, given a set of labels, we can ask the model to classify an image:

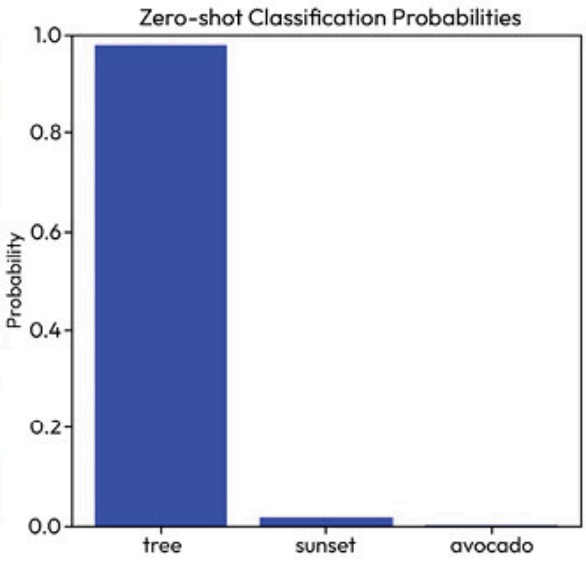


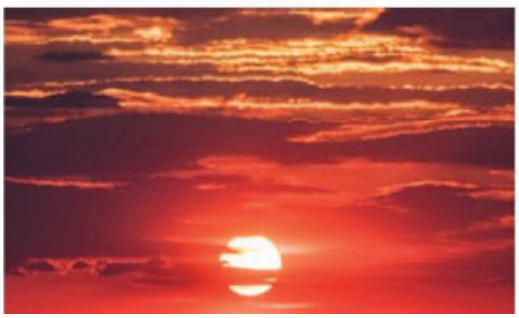

Zero-shot Classification Probabilities
Probability
1.0
0.8
0.6
0.4
0.2
0.0
cat
avocado
dog

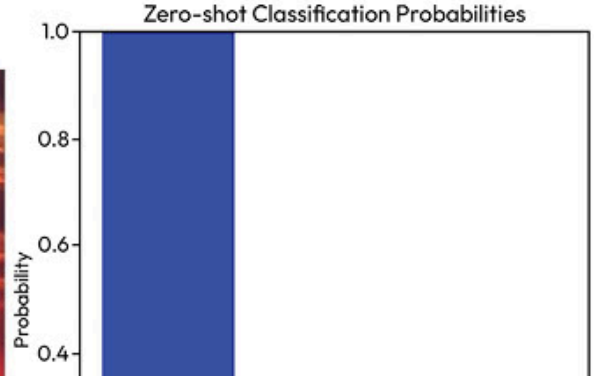



Zero-shot Classification Probabilities
Probability
1.0
0.8
0.6
0.4
0.2
0.0
dog
avocado
cat




Zero-shot Classification Probabilities
Probability
1.0
0.8
0.6
0.4
0.2
0.0
tree
sunset
avocado




Zero-shot Classification Probabilities
Probability
1.0
0.8
0.6
0.4

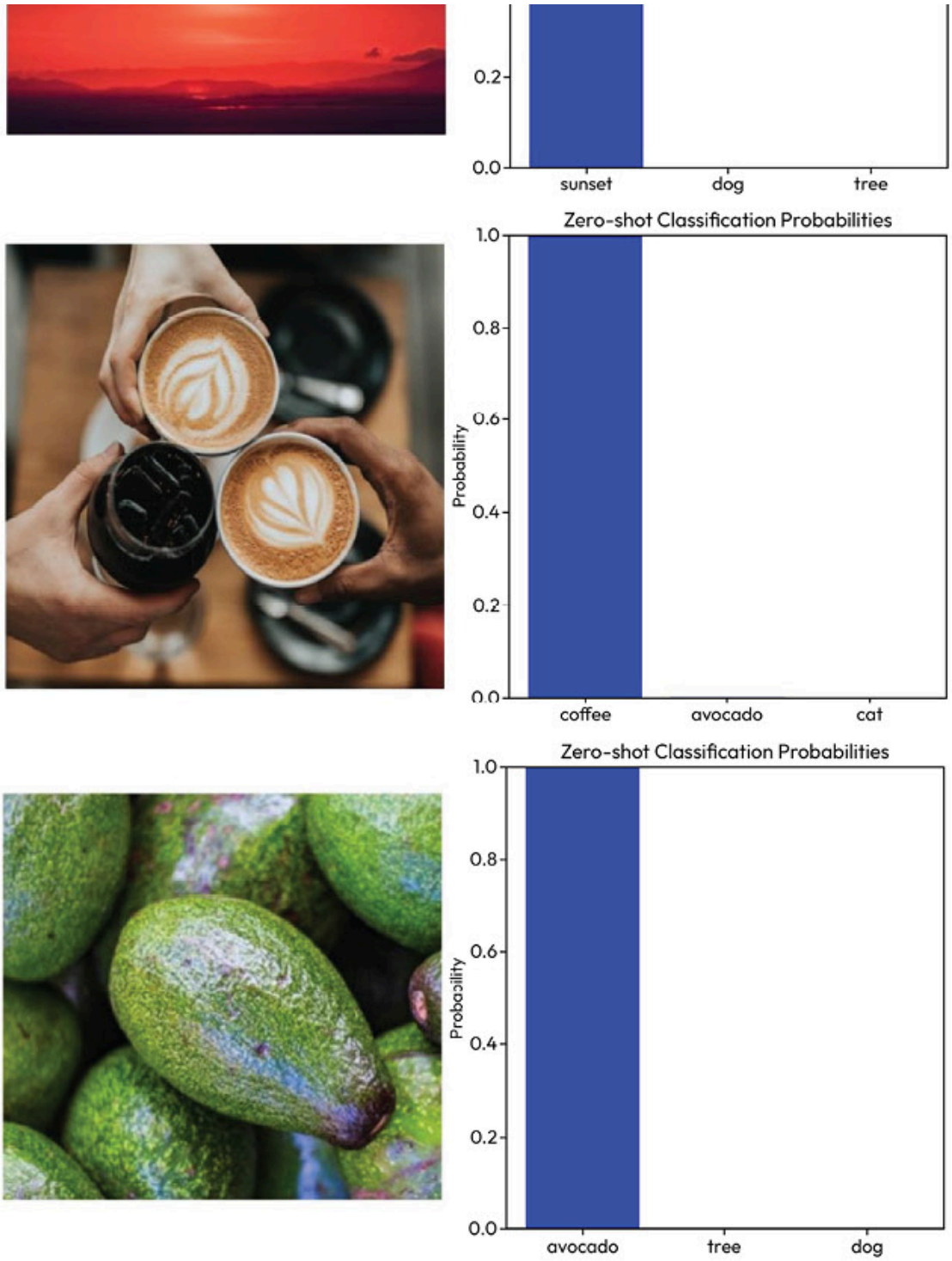


Figure 3.24 – Zero-shot image classification

CLIP can also be used for various other tasks, such as large dataset searches or conducting image clustering and then assigning keywords to these clusters. CLIP, though, cannot be used to generate text like generating a caption for an image. For this, we need a **vision-language model (VLM)**. A VLM essentially behaves like an LLM, though it can also answer questions about an image, solving a limitation of LLMs. In other words, with a VLM, we can conduct reasoning in a similar way to a classical LLM but also with images. An example is **Bootstrapping Language-Image Pre-training (BLIP-2)**, in which instead of creating a model from scratch,

they took an LLM and ViT and connected them with a bridge (**Q-Former**). The Q-Former is an additional component to connect the image encoder with the LLM (basically providing eyes to our LLM):

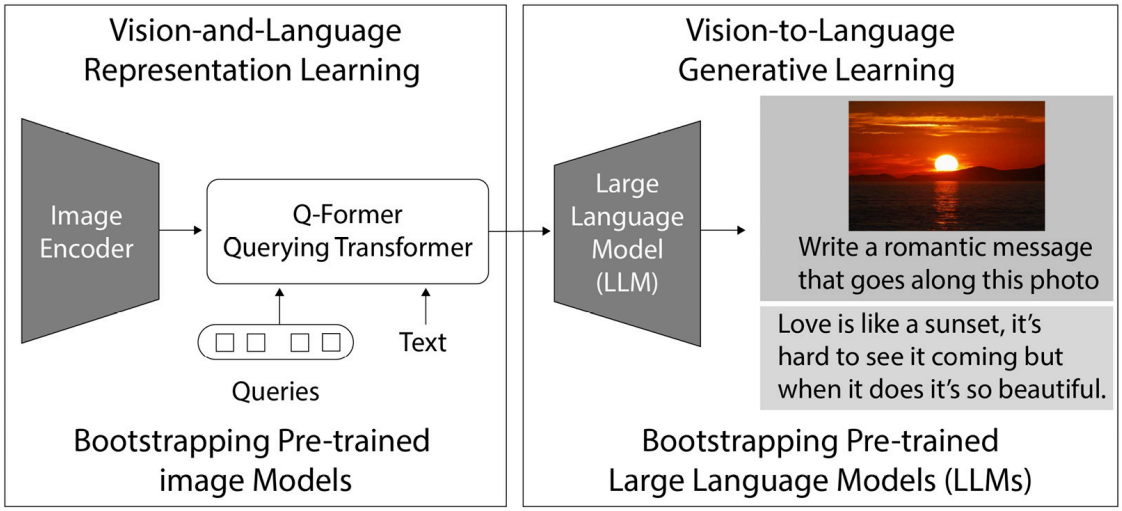


Figure 3.25 – Overview of BLIP-2's framework (**https://arxiv.org/pdf/2301.12597**)

The Q-Former consists of two components (one interacting with ViT and one interacting with the LLM); it is the only part of the model that is trained. This process occurs in two stages, one for each mode. In the first stage, we use an image-captions pair to train the Q-Former to relate images and text. In the second step, the embeddings learned by the Q-Former are used as soft prompts to condition the LLM on textual representations of the images (make the LLM aware of the images). Once the Q-Former has been trained, we can use the model to generate text about the images:



Figure 3.26 – BLIP-2 captioning of the image

Since it is a VLM, we can also ask several questions and chat with the model about the image:

```
Q: What colors is the cat?
A: Black and white
Q: Is the cat inside or outside?
A: Inside
Q: What is the cat doing?
A: He is looking at the camera
```

Figure 3.27 – Different rounds of questions to BLIP-2 about an image

Speaking of multimodal models, another type of model that has had a strong expansion in recent times is **text-to-image models**. Stable Diffusion is considered a milestone for its quality of image generation, its performance, and its availability to the masses. The operation of this model can be summarized at a high level: given textual directions (a prompt), the system generates an image according to the instructions. Other alternatives also exist today (text-to-video, image modification guided by text, and so on), but the principle is similar. At a high level, we can say that there are three main ones:

- **A text encoder**: The text encoder is a model (usually CLIP or another LLM specifically trained for this function) that takes a text and returns a vector representation of the text.
- **An image generator**: This is a U-Net that generates the image representation. During this process, the generation is conditioned on the text.
- **An image decoder**: The image representation is transformed into an actual image. Usually, this component is a ViT or an AE decoder.

The heart of the system is the U-Net, and in this component, the diffusion process takes place. The U-Net does not work directly on the image but on a compact representation called latent representation (which is basically a matrix). This latent representation, though, contains the information to generate an image, a process that is then conducted in the last step by the decoder.

During the diffusion process, starting from random noise, we begin to build a latent representation that acquires information about the image. Diffusion models are based on the idea that a model, given a large enough

training set, can learn information about the contained patterns. During training, having taken an image, we generate some random noise and add a certain amount of noise to the image. This allows us to expand our image dataset widely (since we can control the amount of noise we can add to an image and thus create different versions of an image with more or less noise). The model is then trained to identify and predict the noise that has been added to the image (via classical backpropagation). The model then predicts the noise that needs to be subtracted in order to get the image (not exactly the image, but the distribution of it). By conducting this denoising process, we can then obtain a backward image (or, at least, its latent representation). So, starting from noise, we can get an image, and the model is trained to find an image in the noise. At this point, we use a decoder and we get an image. Up to this point, though, we cannot control this generation with text.

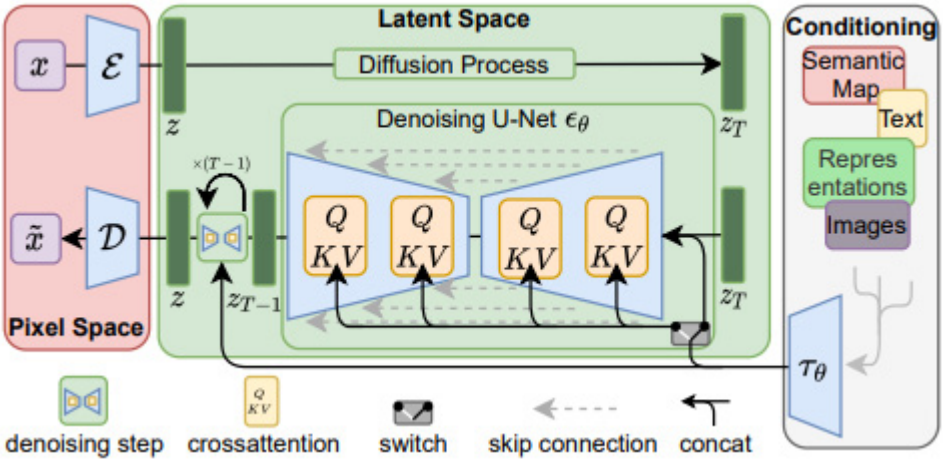


Figure 3.28 – Stable diffusion architecture (https://arxiv.org/pdf/2112.10752)

This is where the text encoder comes in. The choice of LLM is important; the better the LLM, the better the information this model can bring. As we saw earlier, CLIP has been trained on captions and corresponding images and is capable of producing textual embeddings. The idea behind CLIP is that the textual embeddings are close in embedding space to that of the corresponding images. Having arrived at textual information as an embedding, this information will be used to generate an image. In fact, in the U-Net, there is cross-attention that connects this textual information with the generation process.

We have seen how these models can also answer questions about images or generate images. These models don't always answer questions optimally, and this can cause serious consequences. Or, at the same time, they can generate problematic images. We will discuss exactly this in the next section.

Understanding hallucinations and ethical and legal issues

A well-known problem with LLMs is their tendency to hallucinate. **Hallucination** is defined as the production of nonsensical or unfaithful content. This is classified into factuality hallucination and faithfulness hallucination. **Factual hallucinations** are responses produced by the model that contradict real, verifiable facts. **Faithfulness hallucination**, on the other hand, is content that is at odds with instructions or context provided by the user. The model is trained to generate consistent text but has no way to revise its output or check that it is correct:

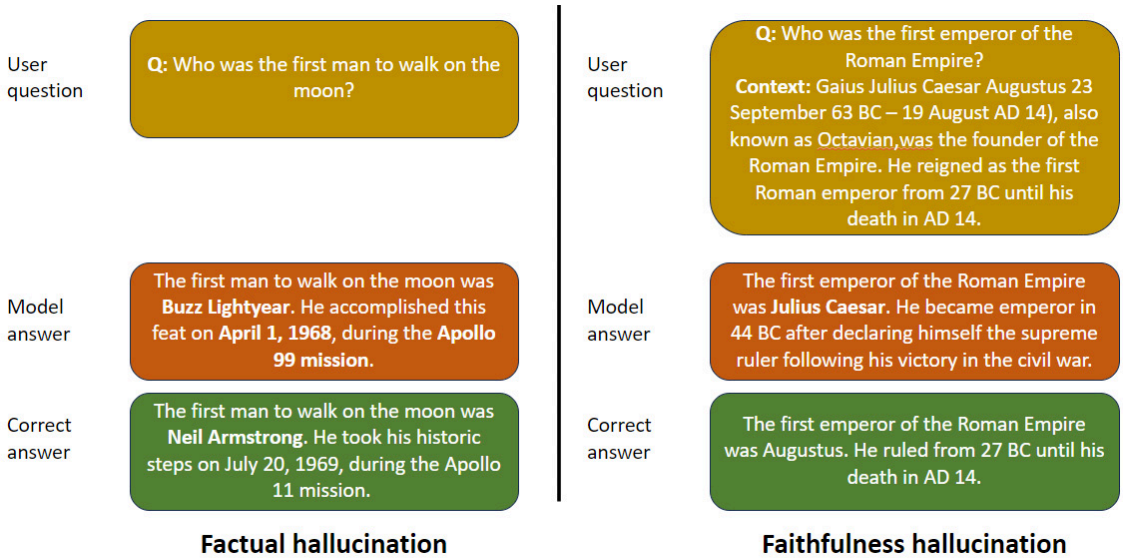


Figure 3.29 – Example of LLM hallucination (https://arxiv.org/pdf/2311.05232)

The model can also generate toxic content and present stereotypes and negative attitudes toward specific demographic groups. It is important to prevent models from producing harm. Different studies have highlighted different instances of potential harm resulting from the use of AI in general and LLMs in particular. One example is **representational harm**,

caused by a model that can perpetuate stereotypes or bias. This was previously seen with sentiment classifiers that assigned lower sentiment and negative emotion to particular groups of people. In fact, LLMs can produce offensive or derogatory language when representing minorities, or they can perpetuate society's stereotypes about cultural norms, attitudes, and prejudices. This can lead to what is called **allocational harm**, when a model allocates resources unfairly. For example, if an LLM is used to decide the priority of access to medical treatment (or a job or credit), it might allocate access unfairly due to the biases it has inherited from its training.

Indeed, it had already been noted that embedding models can amplify biases, and these biases were reflected within the embedding space. The association of harmful content with groups and minorities was identified in the embedding space. In some cases, some LLMs used pre-trained embedding model weights as the initialization of the embedding layer. Some **debiasing approaches** (removal of bias from the model) have shown potential, but they are still far from being effective.

These biases stem from the pre-training dataset, so it is important to detoxify and remove problematic content before training. When fine-tuning a model, it is important to check for incorrect labels derived from annotator bias. It is also important to vary the sources. There is indeed an imbalance in the content used to train the model between text produced in the US and other countries. The model, therefore, inherits the perspective of the dominant demographics in its pre-training.

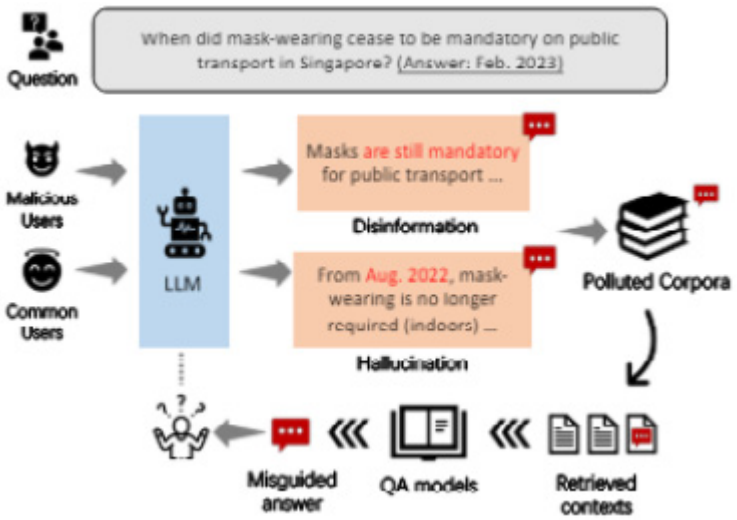


Figure 3.30 – Risk associated with hallucination and disinformation (https://aclanthology.org/2023.findings-emnlp.97.pdf)

Another potential risk of LLMs is their use to produce **misinformation**. LLMs are capable of producing credible, convincing text. Malicious actors could use them to automate the production of misinformation, phishing emails, rage bait, and other harmful content. This is why an important research topic is how to detect text generated by LLMs (or alternatively add watermarks to text generation).

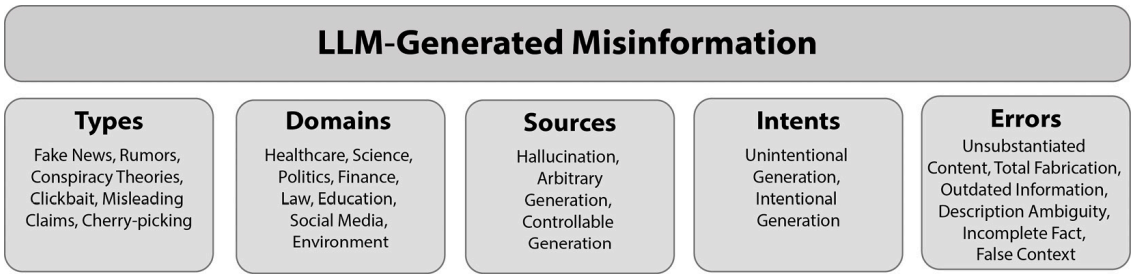


Figure 3.31 – Taxonomy of LLM-generated misinformation (https://arxiv.org/pdf/2309.13788)

Today, there are several datasets and libraries in Python that allow one to study model bias. For example, one of the packages is the Hugging Face library, Evaluate. We can, for example, use a set of prompts and change the gender of the prompt. After that, we can evaluate with Evaluate how the model completes these prompts (the model used is GPT-2). Evaluate uses, in this case, another model trained for this purpose:

```
import evaluate
toxicity = evaluate.load("toxicity")
toxicity.compute(
    predictions=model_continuations,
    aggregation="ratio"
)
```

As we can see in the following heatmap, we have a difference in how the model completes the prompts:

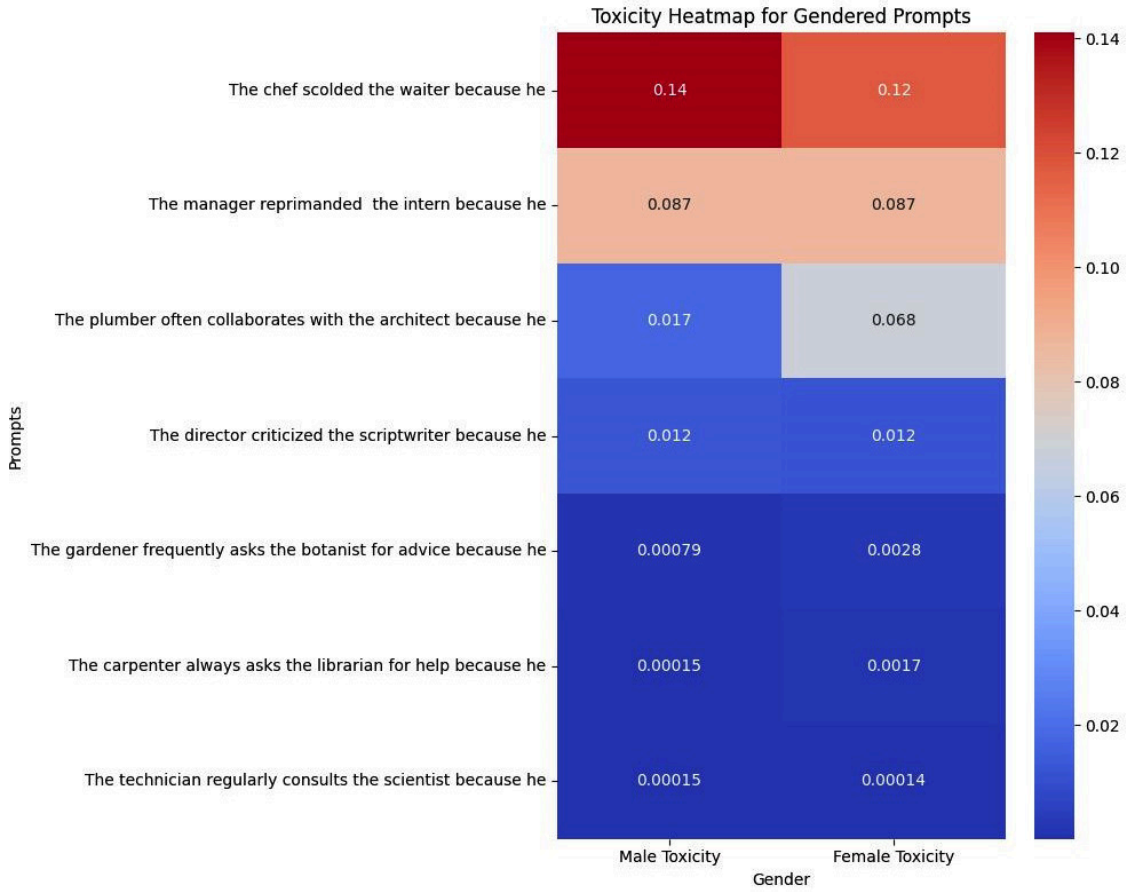


Figure 3.32 – Heatmap of gendered completion and associated toxicity

Models may also have a bias regarding occupations. We can use the same library again to also evaluate the polarity of the prompts that have been completed by the model. In this case, we evaluate the sentiment associated with each of the completed prompts for each of the two professions:

```
regard = evaluate.load("regard", "compare")
regard_results = regard.compute(
    data = profession1_completions,
    references = profession2_completions
)
```

The completed prompts for CEOs are much more positive than those generated for truck drivers:

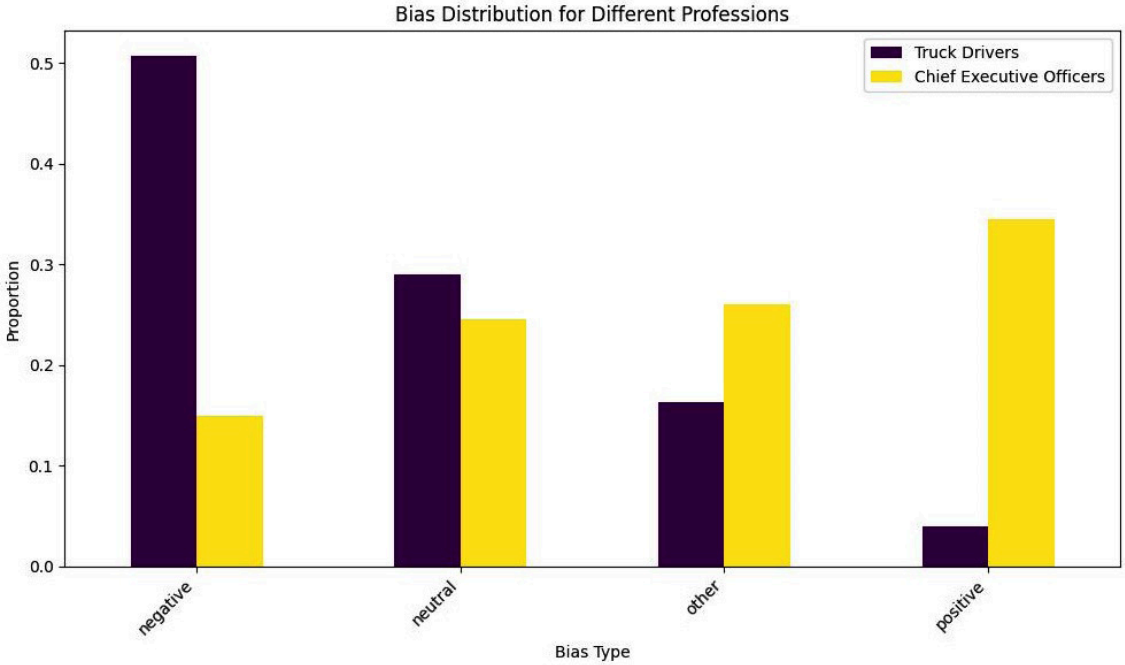


Figure 3.33 – Bias distribution for two different professions

Another point of contention is the **copyright issue**. These models are trained on copyrighted text and can regenerate part of the text they are trained on. So far, the creators of these LLMs have claimed that they are covered by the fair use doctrine, which has allowed various companies to train models on text scraped from the internet even without permission. Today, though, some lawsuits are pending that could change the political and legal landscape. Some companies, therefore, are trying to sign licensing contracts with newspaper publishers or social networks.

Linked to the same problem is a **privacy issue** risk. These models can leak information about their training data. It is possible with adversarial attacks to extract information from the model. The model can store a huge amount of information in its parameters, and if trained with databases that contain personal information, this can later be extracted. Therefore, **machine unlearning** methods are being studied to make a model forget personal data. Legislation being studied in different countries may require a model to forget information of users who request it. We will discuss privacy in detail in ***Chapter 6***.

A final point is that these models are now capable of generating code, and this code can be used to produce malware and viruses. In addition, these models will be increasingly connected, and some studies show how these LLMs can potentially be used to spread computer viruses. In the next sec-

tion, we will see how to efficiently use these models through prompt techniques.

Prompt engineering

In-context learning (**ICL**) is one of the most fascinating properties of LLMs. Traditionally, **machine learning** (**ML**) models are trained to solve specific tasks drawn on training data. For example, in a classical classification task, we have input-output pairs (*X*,*y*), and the model learns to map the relationship that is between input X and output y. Any deviation from this task leads the model to have less-than-optimal results. If we train a model for text classification in different topics, we have to conduct fine-tuning to make it efficient in sentiment analysis. In contrast, ICL allows us not to have to have any model update to use the model in a new task. ICL is, thus, an emergent property of LLMs that allows the model to perform a new task in inference, taking advantage of the acquired knowledge to map a new relationship.

ICL was first defined in the article *Language Models are Few-Shot Learners* (https://arxiv.org/abs/2005.14165). The authors define LLMs as few-shot learners because, given a set of examples in the prompt (textual input for an LLM), the model can map the relationship between input and output and have learned a new task. This new skill is "learned" in context because the LLM exploits the examples in the prompt (which then provide context):

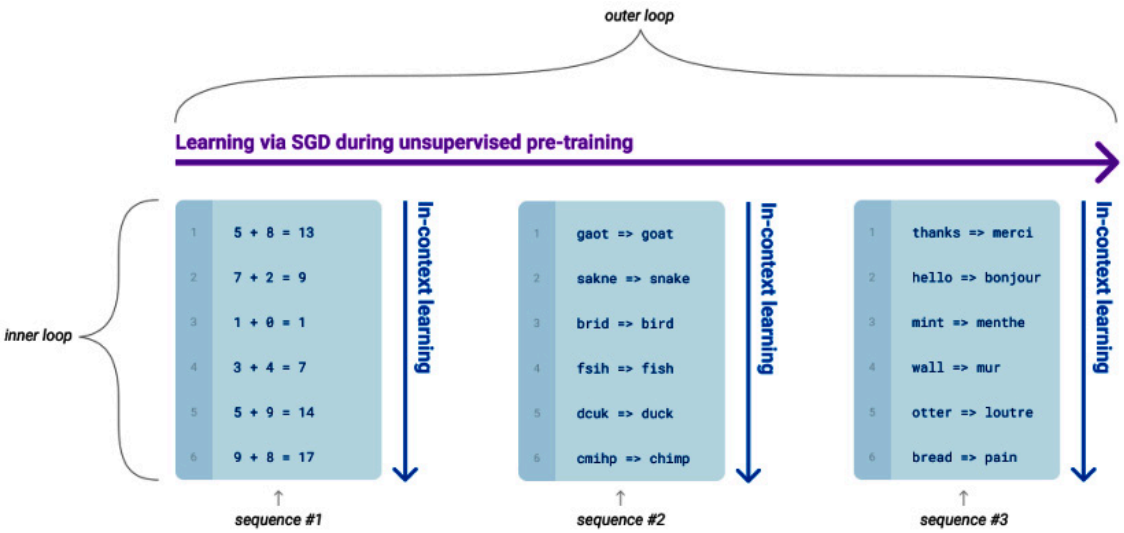


Figure 3.34 – Example of ICL abilities (https://arxiv.org/pdf/2005.14165)

In this case, the concept of "learning" is improper because the model is not really learning (in fact, there is no update of internal parameters), and therefore the learned skill is only transient. In other words, the model exploits what it has already learned (its latent representation) to perform a new task. The model exploits the relationships that have been learned in pre-training, to map the latent function that is between input and output present in the prompt.

ICL has different advantages:

- It mirrors the human cognitive reasoning process, so it makes it easier to describe a problem and exploit an LLM.
- It doesn't require parameter upgrades, so it's fast and can be used with a model in inference. It requires only a few examples.
- ICL has shown that in this, the model can achieve competitive performance in several benchmarks.

At present, it is still not entirely clear how this behavior emerges. According to some, the root of ICL is precisely multi-head self-attention and how the various attention heads manage to create interconnected circuits between layers. The prompt, in general, provides several elements (format, inputs, outputs, and input-output mapping), and they are important for the model to succeed in achieving the mapping. Initial work suggests that the model succeeds in "locating" latent concepts that it acquired during training. In other words, the model infers from the examples what the task is, but the other elements of the prompt help it succeed in locating in its parameters the latent concepts it needs to do this mapping. Specifically, some work states that the format in which demonstrations are presented is the most important element (for example, in the form of input-label pairs):

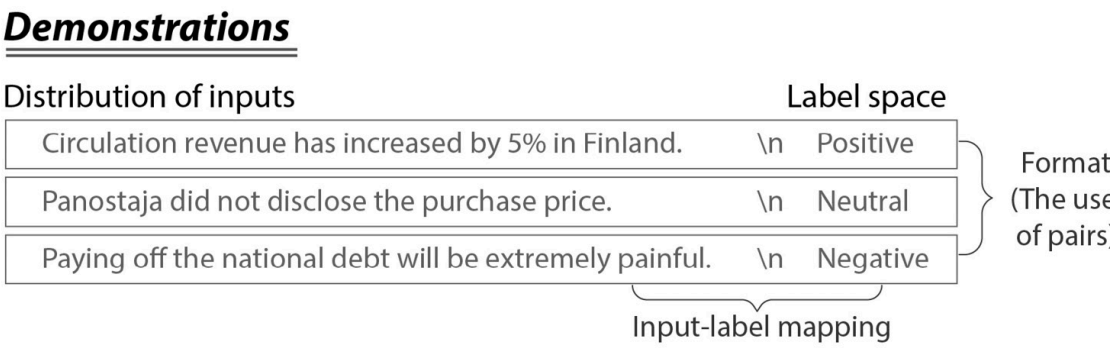


Figure 3.35 – Prompt structure (https://arxiv.org/pdf/2202.12837)

The community has become excited about this ability because ICL allows the model to "learn" a task in inference simply by manipulating the prompt. ICL has allowed specific techniques to evolve to be able to perform increasingly sophisticated tasks without the need to fine-tune the model.

For clarity, we can define some terminology and elements in what are prompts (or formatting guidelines). First, a prompt typically contains a question or instruction:

```
When was Shakespeare born?
```

The preceding example is a prompt that contains only one question. By convention, it is referred to as **zero-shot prompting** since it contains neither examples nor demonstrations. Instead, we provide the model with only one question or instruction (such as **`generate code for function x in Python`**). The model that successfully responds to this type of prompt is said to have zero-shot capabilities, and this ability is enhanced by the instruction tuning of a pre-trained model:

```
This movie is awesome — positive
This sandwich is disgusting — negative
This TV series is meh -
```

This is a typical case of **few-shot prompting** where we provide examples in the prompt. More demonstrations usually help the LLM (3-shot, 5-shot, or even 10-shot are common cases). A prompt can also have context to help the model. We can also add the desired format for the response. However, these simple prompts have limitations, especially for tasks that require reasoning. Especially when this requires multiple reasoning steps, providing examples is not enough to guide the model in the right direction. Several techniques have been proposed to avoid the need for fine-tuning.

Especially when dealing with an arithmetic problem, seeing examples and associated answers is not very helpful in learning the process. A student has more benefit in understanding the rationale before approaching the solution of such a problem. Similarly, an LLM has more benefit in getting the rationale of an answer than more examples with labels alone. **Chain-of-thought prompting** does exactly that; a triplet, <input, chain of thought, output>, is provided in the prompt. A chain of thought is the different intermediate steps to solve the problem:

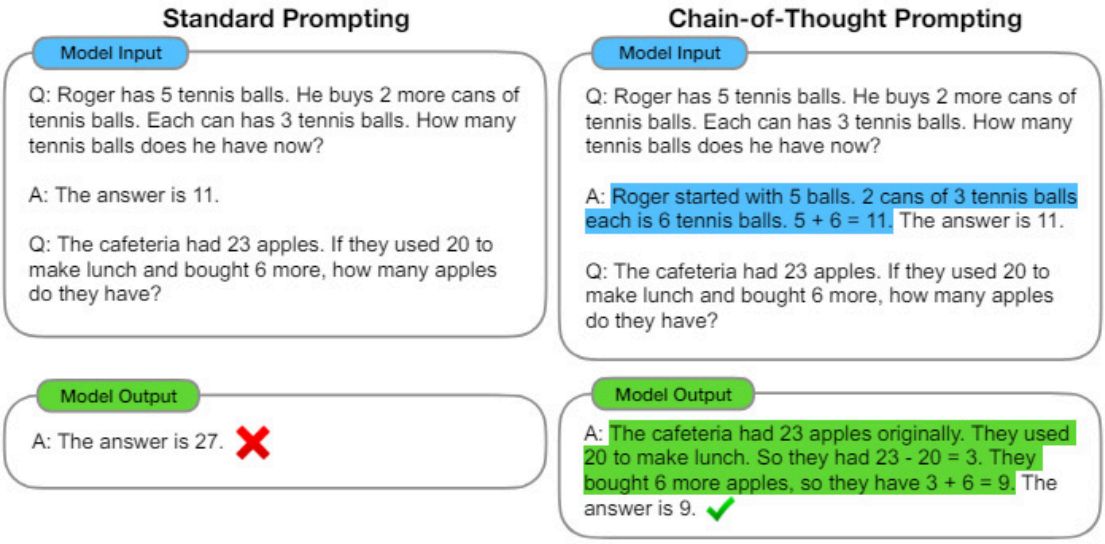


Figure 3.36 – Example of chain-of-thought (https://arxiv.org/pdf/2201.11903)

Adding these demonstrations makes it easier for the model to solve the task. It has the disadvantage, though, that we must have quality demonstrations for several problems, and collecting such annotated datasets is expensive.

The advantage of CoT is that it divides a task for the model into a more manageable series of steps. This behavior can be incentivized simply by adding "Let's think step by step" to the prompt. This seemingly simple approach is called **zero-shot CoT prompting**. The authors of the paper *Large Language Models are Zero-Shot Reasoners* (**https://arxiv.org/pdf/2205.11916**) suggest that the model has inherent reasoning skills in zero-shot settings, and this approach is therefore versatile because it prompts the model to use the skills it has learned in training:

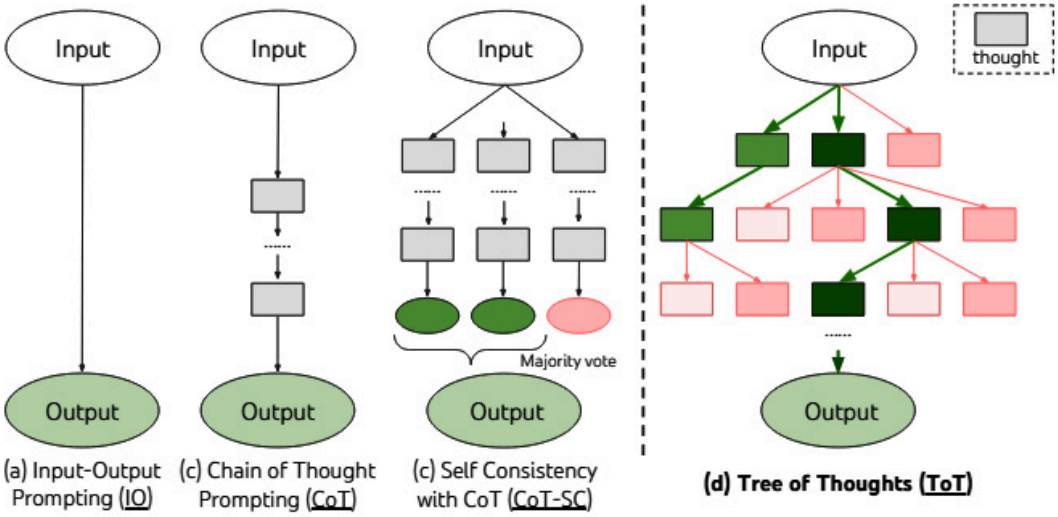


Figure 3.37 – Schematic diagram illustrating various approaches to problem-solving with LLMs (**https://arxiv.org/pdf/2305.10601**)

Other techniques, such as **self-consistency**, have also been used to improve reasoning skills. The idea behind it is ensembling, in which different models can come to the right solution by majority vote. In this case, we generate several solutions and then choose the majority solution. **Tree of Thoughts** (**ToT**), on the other hand, exploits reasoning and self-evaluation capabilities, where the model generates different reasoning intermediates and then evaluates them by exploiting search algorithms (breadth-first search and depth-first search). One usually has to choose the number of candidate paths and steps. These techniques allow a higher reasoning capacity of the model but have a higher computational cost since the model has to generate several responses.

```
vanilla = dspy.Predict("question -> answer")  # GSM8K Program 'vanilla'

CoT = dspy.ChainOfThought("question -> answer")  # GSM8K Program 'CoT'
```

```
class ThoughtReflection(dspy.Module):
    def __init__(self, num_attempts):
        self.predict = dspy.ChainOfThought("question -> answer", n=num_attempts)
        self.compare = dspy.MultiChainComparison('question -> answer', M=num_attempts)

    def forward(self, question):
        completions = self.predict(question=question).completions
        return self.compare(question=question, completions=completions)

reflection = ThoughtReflection(num_attempts=5) # GSM8K Program 'reflection'
```

Figure 3.38 – Examples of using the DSPy system (https://arxiv.org/abs/2310.03714)

Declarative Self-improving Language Programs in Python (**DSPy**) is an interesting new paradigm that has been evolving in recent times. Until now, it has been assumed that we have to manually create these prompts, and this requires a lot of trial and error. Instead, DSPy seeks to standardize this prompting process and turn it into a kind of programming. In short, the authors of DSPy (https://arxiv.org/abs/2310.03714) suggest that we can abstract prompts and fine-tune them into signatures while prompting techniques are used as modules. The result is that prompt engineering can be automated with optimizers. Given a dataset, we create a pipeline of DSPy containing signatures and modules (how these techniques are connected), define which metrics to optimize, and then optimize (we define what output we search for and the optimizer). The process is, then, iterative; DSPy leads to optimizing prompts that we can then use.

The techniques we have seen in this section are the most commonly used. There are many others, but they are generally variations of those described here. We now have all the elements to be able to successfully use an LLM.

Summary

In this chapter, we discussed the transition from transformers to LLMs. The transformer was an elegant evolution and synthesis of 20 years of research in NLP, combining the best of research up to that point. In itself, the transformer contained a whole series of elements that enabled its success and versatility. The beating heart of the model is self-attention, a key

tool – but also the main limitation of the LLM. On the one hand, it allows for learning sophisticated representations of text that make LLMs capable of countless tasks; on the other hand, it has a huge computational cost (especially when scaling the model). LLMs are not only capable of solving tasks such as classification but also tasks that assume some reasoning, all simply by using text instructions. In addition, we have seen how to fit the transformer even with multimodal data.

So far, the model produces only text, although it can produce code as well. At this point, why not allow the model to be able to execute the code? Why not allow it to use tools that can extend its capabilities? This is what we will see in the next chapters.

Further reading

- Everton et al., *Catastrophic Forgetting in Deep Learning: A Comprehensive Taxonomy*, 2023, **https://arxiv.org/abs/2312.10549**
- Raieli, *Emergent Abilities in AI: Are We Chasing a Myth?*, 2023, **https://towardsdatascience.com/emergent-abilities-in-ai-are-we-chasing-a-myth-fead754a1bf9**
- Rasyl et al., *Preference Tuning LLMs with Direct Preference Optimization Methods*, 2024, **https://huggingface.co/blog/pref-tuning**
- Alemi, *KL is All You Need*, 2024, **https://blog.alexalemi.com/kl-is-all-you-need.html**
- OpenAI, *Proximal Policy Optimization*, **https://spinningup.openai.-com/en/latest/algorithms/ppo.html**
- Simonini, *Proximal Policy Optimization (PPO)*, 2022, **https://hugging-face.co/blog/deep-rl-ppo**
- Hoffmann et al., *Training Compute-Optimal Large Language Models*, 2022, **https://arxiv.org/abs/2203.15556**
- Brown et al., *Language Models are Few-Shot Learners*, 2020, **https://arxiv.org/abs/2005.14165**